

# SAGE: Quorum-Certified Migration Between Heterogeneous Consensus Engines for Permissioned Blockchains

Anonymous Authors

**Abstract**—Migrating a live permissioned blockchain from Proof-of-Authority to HotStuff BFT risks ledger divergence because their finalization rules differ. SAGE avoids a scheduled halt by retaining legacy authority until an  $n - f$  quorum certifies the boundary for authority transfer. A quorum-certified manifest binds boundary, epoch, engines, and replay context under a durable one-body-per-epoch lock; rollback requires its matching certificate before the manifest-bound deadline. Committee-wide fence deployment and terminal-certificate dissemination are specified but remain unimplemented in this research prototype. Evaluation used bounded TLC, a loopback TCP testbed, and six hosts in one cloud region and availability domain. For  $n \in \{4, 7, 10\}$  TLC found no counterexample to cross-boundary decision uniqueness, complete-manifest agreement, or certificate-gated terminal transitions, and falsified every paired broken control. Under a balanced partition at cutover SAGE’s gate stayed closed with migration success 0: gate-closure evidence, not migration liveness. The same schedule forked an unsafe  $q = 2f + 1$  control in 20/20 trials, while  $q = n - f$  and the safe interval’s minimum each recorded 0/20, consistent with the algebraic result that  $n - f$  is conservative rather than uniquely safe. Crash-restart passed four of five seeds, then five of five in a follow-up campaign, with one open pacemaker fault. Physical geo-WAN behavior is not established, and no cited comparator or external-client benchmark was executed.

**Index Terms**—Consensus migration, Byzantine fault tolerance, permissioned blockchain, live upgrade, state machine replication, quorum intersection, bounded rollback, multi-signature certificates.

## I. INTRODUCTION

**P**ERMISSIONED blockchains often begin with a simple authority-based consensus engine and later require stronger Byzantine fault tolerance, higher throughput, or a different finality mechanism. Replacing the engine after the chain is operational is not an ordinary software upgrade. The old and new engines may use different quorum families and different rules for deciding when a block is irreversible. If validators switch independently, a network partition can leave both engines finalizing incompatible histories.

Constructed controls in Section VII demonstrate that independent switching can leave both engines finalizing incompatible histories. Stop-the-world migration preserves a clear boundary but interrupts the model by an assigned pause; the reported 8 ms value is a simulator input, not measured client downtime. Membership reconfiguration and deployed transitions address related but not identical settings [26], [27], [34], [35], [10], [12], [7], [13]. We do not infer a general capability gap without a direct mechanism-level comparison

to Abstract/BFTBrain, Cox, and deployed transition protocols. Adaptive systems choose or switch protocols under their own state-transfer and epoch models [15], [32], [28], [29], [30], [31]. SAGE’s candidate residual distinction is a separately  $n - f$ -certified boundary root plus explicit provisional finality and rollback; this distinction still requires direct comparison with BFTBrain/Abstract and Cox.

The prototype passes PoA→HotStuff authority after a persisted multi-share CutCert. During dual-run, the legacy engine is the only authoritative finalizer and the target validates the same finalized blocks without proposing. The CutCert quorum-binds the chain, epoch, configuration, target generation, boundary block, and root. Independently, an  $n - f$  ManifestCertificate under the Manifest authority scope quorum-authenticates the complete manifest payload, and the local store enforces a durable one-body lock per epoch. Dedicated Terminal-scope Abort/Seal certificates, durable exclusive terminal signing, and certificate-gated local transitions are also implemented. Reliable manifest and terminal-share gossip, committee-wide fence deployment, and terminal-round convergence are not implemented in the current runtime; bounded post-heal cutover convergence is measured in Section VII-C.

The central observation is that a target engine’s ordinary commit quorum need not be a safe migration quorum. For  $\mathcal{V}_M = \mathcal{V}_1 = \mathcal{V}_2$ , with one corruption universe containing at most  $f$  Byzantine members and  $n \geq 3f + 1$ , a certificate threshold  $q$  is live with  $f$  silent validators when  $q \leq n - f$ , and conflicting certificates necessarily share a correct signer when  $2q - n > f$ . Thus every integer threshold in  $((n + f)/2, n - f]$  is safe and live under the fixed-committee assumptions. SAGE chooses  $n - f$ , the maximal live threshold, rather than treating the target engine’s native quorum as automatically sufficient at a heterogeneous boundary.

This paper makes four contributions:

- 1) It formulates migration between deterministic-finality engines through an abstract engine interface and a single-authority shadow-execution model.
- 2) It specifies a CutCert boundary, a quorum-authenticated complete-manifest envelope, provisional labels, and dedicated  $n - f$  Abort/Seal certificates with a durable exclusive signing predicate and certificate-gated terminal transitions.
- 3) It derives the CutCert quorum algebra analytically and bounded-checks complete-manifest agreement, a manifest-bound fence action, and certificate-gated absolute promotion, each paired with a falsifying control

the checker must catch.

- 4) It evaluates the design through simulation, a TCP multi-process testbed, bounded model checking with falsifying controls, and a retained configured-endpoint campaign. Each evidence tier is reported according to what its archived bytes can establish.

a) *Scope.*: SAGE targets monolithic permissioned chains with deterministic finality and a fixed migration committee. It inherits the safety assumptions of both engines and does not repair an equivocating legacy protocol. Clean transaction replay requires metadata-independent execution or a committed replay context. Probabilistic-finality sources, concurrent membership reconfiguration, sharded state, arbitrary external nondeterminism, and a mechanized refinement proof from the formal model to the executable system are outside the present scope. The implementation is a research prototype rather than a production blockchain node.

## II. RELATED WORK

a) *Consensus and reconfiguration.*: Classical results establish the limits of asynchronous consensus [24] and the partial- synchrony model used by practical BFT protocols [23], [22]. Modern engines include HotStuff [1], Tendermint [7], DAG-based protocols [2], [3], [4], [33], [6], [5], and broadcast/consensus hybrids such as Sui Lutriss [9]. Shoal++ is a current high-throughput, low-latency DAG-BFT design, not a runtime-switching baseline [33]. Reconfiguration changes the replica set while retaining a consensus rule [26], [27], [21]. The evaluated SAGE artifact studies a fixed- committee PoA→HotStuff boundary. Its relation to current runtime-switching frameworks is qualitative here and must be established against BFTBrain/Abstract and Cox.

b) *Deployed transitions.*: The Ethereum Merge combined a long-running parallel proof-of-stake chain with a fixed terminal-total-difficulty trigger [34], [35]. Ethereum Gasper, Polkadot GRANDPA, Cosmos/Tendermint, and Tezos Tenderbake illustrate deployed finality mechanisms with different protocol structures [10], [12], [7], [13]. The prototype adds a migration- specific CutCert, complete-payload ManifestCertificate, and provisional target labels; reliable manifest and terminal-share dissemination plus live terminal-round convergence remain open.

c) *Runtime and adaptive switching.*: Abstract’s “The Next 700 BFT Protocols” composes protocol instances through an Abstract/Backup wrapper, idempotent requests, and initialization histories [15]. BFTBrain is the principal recent runtime-switching comparator: within one BFT cluster it selects among PBFT, Zyzzyva, CheapBFT, Prime, SBFT, and HotStuff-2 at epoch boundaries, using checkpoint/init histories and an Abstract-derived switching argument [32]. Cox coordinates homogeneous-BFT switching using configuration blocks, stable-checkpoint state, core initialization, and chained recovery semantics [14]. Other adaptive frameworks select or tune protocols under changing conditions [16], [17], [19], [20], [28], [29], [30], [31]; resource- and attack-resilient designs harden a protocol family [18], [36], [37], [38]. Table I compares mechanisms rather than headline labels. BFTBrain and

Cox were not executed in this work; the empirical checkpoint-style and under-quorum arms are internal controls, not implementations of either system.

d) *Positioning.*: Permissioned platforms such as Hyperledger Fabric [8] provide modular execution and consensus components. The current artifact implements the CutCert-gated handoff subset; integration with selectors and complete terminal recovery remains future work. It additionally implements a separately  $n - f$ -certified heterogeneous boundary root, explicit provisional finality, and Terminal-scope Abort/Seal certificate gates. Reliable manifest and terminal-share dissemination, live committee-wide fence deployment, and a direct BFTBrain/Abstract/Cox evaluation are still required before claiming that this combination is a settled advance. The present scope is deterministic-finality, monolithic-state migration under one fixed identical committee.

## III. SYSTEM MODEL AND ENGINE INTERFACE

### A. Network and Validators

A permissioned blockchain is maintained under the partial synchrony model [23]: there is an unknown global stabilization time (GST) after which message delay is bounded by  $\Delta$ ; before GST delays are arbitrary. For analysis we assume partial synchrony and authenticated channels. The executable testbed uses real TCP and testbed Ed25519 shares but does not implement production transport authentication or key custody. This paper’s theorems and artifact require  $\mathcal{V}_M = \mathcal{V}_1 = \mathcal{V}_2$ ,  $|\mathcal{V}_M| = n$ , no membership changes during  $[h_d, h_r]$ , and one corruption set of size at most  $f \leq \min(f_1, f_2)$ . Distinct-committee migration requires a future joint-quorum and key-mapping model and is outside the present guarantees. The adversary may delay messages before GST and induce crashes and partitions in the one committee.

### B. Chain State and Block Transition

Let the consensus-independent replicated state at height  $h$  be

$$X_h = (A_h, P_h), \quad r_h = \text{root}(X_h), \quad (1)$$

where  $A_h$  is the account and storage state of the execution layer,  $P_h$  is platform-managed state including the validator registry and the migration schedule. Each engine also maintains local consensus metadata  $C_h^{(j)}$ , which is deliberately excluded from the cross-engine root because heterogeneous engines encode different views, locks, and certificates. A block  $B_h$  carries an ordered transaction list and a header. Execution applies a deterministic transition

$$X_h = \delta_{\text{exec}}(X_{h-1}, B_h), \quad (2)$$

and the header commits the post-execution state root  $r_h$ . We adopt the post-state convention throughout:  $r_h$  is the root after applying  $B_h$ . A block is canonical only if it additionally satisfies the active engine’s finalization rule, defined next. Table II lists the notation.

TABLE I

MECHANISM-SCOPED COMPARISON OF RUNTIME SWITCHING. “NOT REPORTED” MEANS THAT THE CITED MECHANISM DOES NOT EXPOSE THE SAGE-SPECIFIC SURFACE IN ITS STATED MODEL, NOT THAT THE SYSTEM IS UNSAFE. BFTBRAIN AND COX ARE LITERATURE-ONLY COMPARATORS AND WERE NOT EXECUTED IN THIS WORK.

| System          | Committee model                                         | Source $\rightarrow$ target fault-model crossing                                                     | State-transfer / handoff object                                                                                        | Switch-agreement mechanism                                                                        | Client-visible finality tiers                                   | Rollback semantics                                                   | Formal scope                                                                                                     | Evidence type                                                         |
|-----------------|---------------------------------------------------------|------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|----------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------|
| SAGE            | One identical fixed committee in present theorems       | PoA $\rightarrow$ HotStuff under one corruption universe; distinct sets are future work              | Separately $n - f$ -certified boundary root plus $n - f$ complete-payload. ManifestCertificate; manifest gossip absent | Independent $n - f$ cutover, manifest, abort, and seal certificates; terminal-share gossip absent | Internal provisional label and seal gate; no client flag        | Certificate-gated local rollback; no live network terminal round     | CutCert algebra plus bounded boundary, manifest, fence, and terminal-gate TLC; live-host fence deployment future | Prototype component, simulator, process, and bounded-model evidence   |
| BFTBrain [32]   | Same BFT cluster with epoch switching                   | Switches among supported BFT instances; no unequal source/target corruption-universe claim used here | Stable checkpoint and initialization history through Abstract-derived wrappers                                         | Epoch switch decision plus wrapper protocol                                                       | No SAGE-style provisional/sealed interface reported             | Abstract/Backup-style recovery, not deadline-bounded client rollback | Safety/liveness argument for supported wrapped protocols                                                         | Published multi-machine system and public artifact; not executed here |
| Abstract/Backup | [15] One replica group composing BFT protocol instances | Common BFT fault model; no heterogeneous fault-universe crossing used here                           | Initialization history and idempotent request state                                                                    | Abstract wrapper with Backup for safe composition/progress                                        | Single application history; no provisional/sealed tier reported | Backup fallback/recovery, not client-visible suffix rollback         | Idempotency and composition theorem in the Abstract model                                                        | Formal framework and reported evaluation; not executed here           |
| Cox [14]        | Homogeneous BFT setting                                 | No unequal source/target fault-model crossing used here                                              | Configuration block, stable checkpoint, core initialization                                                            | Stable-checkpoint-coordinated switch                                                              | No SAGE-style provisional/sealed interface reported             | Chained recovery semantics; no SAGE deadline model                   | Safety/liveness in the Cox system model                                                                          | Published design/evaluation; not executed here                        |

### C. Abstract Engine Interface

To state SAGE’s generality precisely, we abstract a consensus engine as an interface rather than fix it to one protocol.

**Definition 1** (Consensus engine). A consensus engine  $\mathcal{C}$  is a tuple  $(Q, \text{Final}, \text{Root}, \text{Init})$  where  $Q$  is a finalization quorum rule,  $\text{Final}$  is a single-finalizer predicate that finalizes at most one block per height when a  $Q$  quorum endorses it,  $\text{Root}$  is a deterministic state-root function, and  $\text{Init}$  initializes the engine’s internal consensus metadata  $C_h^{(j)}$  from a state root and a height. An engine is BFT if  $Q$  requires  $2f + 1$  of  $n \geq 3f + 1$ ; it is an authority engine if  $Q$  requires an honest majority  $\lfloor n/2 \rfloor + 1$ .

This is an interface contract; conformance of an external engine is an assumption. The single-finalizer predicate  $\text{Final}$  requires *deterministic* finality: a finalized block is irreversible under the engine’s own rule. This deliberately excludes *probabilistic*-finality legacy engines (e.g. longest-chain Nakamoto consensus), whose “finality” is a confirmation-depth confidence rather than a decision. SAGE is scoped to a deterministic-to-deterministic crossing (authority/BFT  $\rightarrow$  BFT); migrating *from* a probabilistic-finality source would require first defining a deterministic checkpoint over the probabilistic prefix (as the Ethereum Merge did with a terminal-total-difficulty trigger and a finality gadget), which is a separate problem we do not address. Applicability additionally requires compatible deterministic state execution, a shadow-validation adapter, externally bootstrappable target metadata, durable generation fencing, and a fixed registered migration committee. Without these capabilities, the shadow-anchoring invariant or certified handoff is not well-defined. Accordingly, the legacy engine  $\mathcal{C}_1$  and target engine  $\mathcal{C}_2$  must conform to Definition 1, expose a deterministic root, admit a shadow-usable validation adapter, and satisfy the fencing and bootstrap obligations above. The artifact instantiates PoA  $\rightarrow$  HotStuff and exercises a Raft  $\rightarrow$  HotStuff component path. General deterministic-engine pairs are supported only if they supply the listed adapters and fencing obligations; this has not been demonstrated.

**Definition 2** (Target bootstrap). At cutover height  $h_c$  the target

engine initializes its internal consensus metadata by  $C_{h_c}^{(2)} \leftarrow C_2.\text{Init}(r_{h_c-1}, h_c)$ , that is, the target adopts the legacy-finalized state root as its genesis anchor, treats the bootstrap certificate carried by the manifest as an authority-transfer/genesis anchor without installing it as an ordinary target-engine QC, and starts its first view at  $h_c$ . In the HotStuff instantiation, the highest QC and lock are initially empty; the first target-native QC must be formed by a real target quorum. The execution state  $A$  and platform state  $P$  are carried unchanged; only the target-local  $C^{(2)}$  is initialized.

Definition 2 makes explicit the step that a heterogeneous handoff requires and that a homogeneous reconfiguration does not: the target engine’s consensus metadata has no history on the legacy chain and must be seeded from the boundary anchor.

### D. Combined Fault Threshold During Migration

The present theorems use one identical committee and one corruption universe. PoA is safe under an honest majority,  $Q_1 = \lfloor n/2 \rfloor + 1$ , with its declared legacy-phase assumptions, whereas HotStuff requires  $Q_2 = 2f_2 + 1$  with  $n \geq 3f_2 + 1$ . We assume at most  $f \leq \min(f_1, f_2)$  Byzantine validators in that same committee and require  $n \geq 3f + 1$  for the CutCert threshold interval. Distinct committee sizes, corruption identities, and cross-set key mappings are outside all analytical guarantees in Section VI. A supplementary bounded model (`formal/SageDistinctCommittee.tla`) relaxes the identity restriction: with a migration committee  $V_M$  and a target committee  $V_T$  that partially overlap or are disjoint, TLC finds no counterexample to boundary agreement or cutover-certificate uniqueness when the gate uses the conservative joint threshold  $\max(|V_M| - f_M, |V_T| - f_T)$ , while a control that sizes the gate against the smaller committee alone is falsified. That result is a bounded model check, not a deployment claim: the runtime still assumes one identical committee, so a joint-quorum theorem and runtime role/key registry remain future work.

*What the legacy engine tolerates during dual-run.* We state the PoA fault model precisely, because “PoA safety” is often left informal. A standard round-robin PoA engine provides safety under an *honest majority* and tolerates  $f_1 < n/2$  crash

TABLE II  
SUMMARY OF NOTATION.

| Symbol                                          | Description                                                                      |
|-------------------------------------------------|----------------------------------------------------------------------------------|
| $\mathcal{V}_1 = \mathcal{V}_2 = \mathcal{V}_M$ | one identical legacy, target, and migration committee                            |
| $n$                                             | size of the identical committee during the fixed migration epoch                 |
| $f_1, f_2$                                      | Byzantine threshold of $\mathcal{C}_1$ (PoA) and $\mathcal{C}_2$ (HotStuff)      |
| $f$                                             | one corruption set of size at most $f \leq \min(f_1, f_2)$                       |
| $\Delta$                                        | post-GST message delay bound                                                     |
| $h$                                             | block height                                                                     |
| $X_h, r_h$                                      | consensus-independent replicated state and its post-execution root at height $h$ |
| $\mathcal{C}_h^{(j)}$                           | engine- $j$ local consensus metadata, excluded from $r_h$                        |
| $\mathcal{C}_1, \mathcal{C}_2$                  | legacy and target consensus engines                                              |
| $\mathcal{Q}_i$                                 | finalization quorum of engine $\mathcal{C}_i$                                    |
| $h_d, h_c, h_r$                                 | dual-run start, cutover, rollback-deadline heights                               |
| $\sigma_h, \tilde{r}_h$                         | shadow verdict and shadow-recomputed root at height $h$                          |
| $\kappa$                                        | shadow-stability threshold (consecutive valid blocks)                            |
| $\mathcal{M}, \pi_B, \pi_M$                     | manifest, boundary CutCert, and complete-payload ManifestCertificate             |
| CutCert, AbortCert,                             | quorum certificates for cutover, abort, and seal;                                |
| SealCert                                        | all use multi-share quorum evidence                                              |
| $\mathcal{A}(h)$                                | authoritative engine at height $h$                                               |
| $\tau$                                          | target-engine progress timeout for the abort path                                |

faults, but it does *not* by itself prevent a Byzantine authority from equivocating (signing two distinct blocks at one height): with no  $2f+1$ -style quorum intersection, PoA's one-block-per-height guarantee relies on authorities not equivocating. We therefore make the assumption explicit and offer two readings. (i) *Crash-only legacy model*: if the deployment trusts its authorities not to equivocate, then during dual-run SAGE inherits that crash-fault model; the single corruption set remains bounded by  $f \leq \min(f_1, f_2)$ . (ii) *Byzantine legacy model*: if authority equivocation must be tolerated, the legacy engine must either enforce non-equivocation for every authority key (e.g. attested append-only key custody) or require a finalization quorum  $q_1$  satisfying  $2q_1 - n > f_1$ . A bare  $\lfloor n/2 \rfloor + 1$  signature quorum is insufficient because two such quorums can intersect only in Byzantine signers. The evaluated legacy phase assumes PoA authorities do not equivocate. Byzantine-safe legacy operation requires an external non-equivocation mechanism or a stronger legacy quorum, neither implemented here. The conditional cross-boundary argument therefore requires the legacy engine to satisfy its declared single-finalizer predicate during  $[h_d, h_c)$ .

*Degraded-threshold window*. During dual-run, safety is bounded by the weaker legacy engine. This is the legacy chain's pre-existing bound: a chain already running on  $\mathcal{C}_1$  tolerates at most  $f_1$  faults at all times, so an adversary able to compromise more than  $f_1$  PoA validators can already violate that chain's safety irrespective of any migration. SAGE adds no finalizing authority during dual-run. The window is

TABLE III  
CRYPTOGRAPHIC ASSUMPTIONS AND THE PROPERTY EACH UNDERWRITES.

| Assumption                                           | Underwrites                                                                                        |
|------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| Collision-resistant $\text{root}(\cdot)$ and $H$     | boundary-state and complete-manifest binding (Thm. 6)                                              |
| EUFCMA signatures                                    | CutCert, ManifestCertificate, AbortCert, and SealCert share authenticity                           |
| Chain-id and epoch binding                           | boundary, envelope, and terminal-payload replay resistance (Thm. 7)                                |
| $n \geq 3f + 1$ in one identical migration committee | CutCert, ManifestCertificate, and terminal-certificate quorum intersection (Lem. 1, Thms. 1 and 6) |

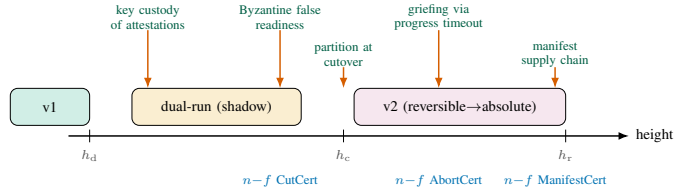


Fig. 1. Attack surfaces across the migration timeline. The implemented evidence covers the  $n - f$  CutCert gate, complete-manifest quorum acceptance, durable local manifest and terminal locks, certificate-gated Abort/Seal transitions, and a local restart-safe fence predicate. Full live-host fence deployment plus manifest and terminal-share dissemination remain open. A future per-epoch cap would limit accepted rollback cycles but would not guarantee terminal progress when no certificate forms.

also *bounded and operator-controlled*: it spans only  $[h_d, h_c)$ , whose length is a governance parameter that can be made as short as the stability threshold  $\kappa$  requires. During this window the target engine performs independent shadow validation of every block and recomputes its own root, which provides an *additional* detection surface for invalid or semantically incompatible execution. A well-formed malicious legacy block that both engines execute identically is not detected merely by matching shadow roots. SAGE therefore neither raises the fault bound it inherits from  $\mathcal{C}_1$  nor prolongs it indefinitely. The analysis does not cover strategic compromise timing within this interval.

#### IV. THREAT MODEL AND ABORT ANALYSIS

Under the present theorem scope, one identical committee has at most  $f$  Byzantine members; partitions and crashes are adversarially scheduled, and messages may be delayed before GST. The adversary cannot forge signatures or find collisions in  $\text{root}(\cdot)$  or the manifest hash  $H$ . Fig. 1 places the attack surfaces on the migration timeline, and Table IV summarizes the component triggers and future coordination requirements.

*Partition at cutover*. The implemented  $n - f$  gate prevents a side below threshold from activating target authority, so that side stalls locally. Under the identical-committee scope, quorum intersection permits at most one certified boundary tuple. Post-heal cutover convergence is observed in the five-seed multi-process campaign and same-region repetition in Section VII-C; reliable certificate retransmission and adoption under general schedules remain unimplemented. *Byzantine false readiness*. In the one implemented equal-power committee, a CutCert has  $n - f$  signatures and contains at least  $f + 1$

TABLE IV  
ABORT-COMPONENT TRIGGERS AND COORDINATION STATUS. LOCAL  
DETECTION DOES NOT CHANGE AUTHORITY; AN  $n - f$  ABORTCERT  
GATES ROLLBACK, BUT NETWORK SHARE DISSEMINATION IS NOT  
IMPLEMENTED.

| Trigger                  | Who can invoke                       | Guarantee                                                                                |
|--------------------------|--------------------------------------|------------------------------------------------------------------------------------------|
| Progress timeout $\tau$  | any (incl. $f$ Byzantine stalling)   | bounded delay $\leq \rho(\tau + T_{\text{recover}})/\text{epoch}$ , not denial (Prop. 1) |
| Quorum abort ( $n - f$ ) | needs $\geq f + 1$ correct           | cannot be forced by $f$ Byzantine                                                        |
| Manifest mismatch        | detected locally by any correct node | tamper-evident (Thm. 6)                                                                  |

correct signers; the runtime additionally checks the certified boundary against its local committed tip. This CutCert-side property does not establish complete-manifest agreement by itself; the independent ManifestCertificate supplies that local acceptance rule. *Progress grieving*. The component exposes local abort triggers and a deadline guard. It defines and verifies  $n - f$  AbortCerts, durably locks each validator to one terminal payload, and gates rollback on the certificate. A future network protocol must disseminate and gather shares, coordinate a fresh legacy generation, and enforce any per-epoch retry cap. The accounting in Proposition 1 is conditional on those future mechanisms. *Manifest supply chain*. In the manifest component verifier, the CutCert prevents a tool from changing the certified boundary root, while the independently verified ManifestCertificate binds the complete signing payload, including the tool hash and replay-context root. This is quorum agreement on the hash, not an endorsement that the tool is safe. Production deployment therefore still requires an allowlist or reproducible-build policy for acceptable tooling.

*Rollback replay mismatch*. The rollback component refuses incomplete or mismatched context while retaining the anchor, and the controller rejects rollback without a valid  $n - f$  AbortCert. Network dissemination, committee-wide restoration, and client-visible recovery are not implemented. Root-equivalent replay remains conditional on Assumptions 2 and 3; a refused suffix requires application-specific operator recovery rather than acceptance of an unaudited state.

## V. THE SAGE PROTOCOL

SAGE (Shadow-Anchored Graceful Evolution) realizes a heterogeneous engine migration through four mechanisms: a single-finalizer dual-run invariant, a quorum-certified cutover decision, an  $n - f$  multi-signature migration manifest, and a deadline-bounded rollback under a two-tier finality model. Fig. 2 shows the data path, Fig. 3 the temporal message flow, and Fig. 4 the controller as a labeled transition system.

### A. Migration Schedule as a Transition System

A migration is governed by three heights  $h_d < h_c \leq h_r$  fixed in platform state  $P$  and amendable only through the legacy engine before  $h_d$ . Rather than a piecewise height function with overlapping cases, we define the active engine by the labeled transition system of Fig. 4, with states  $\{V1, DUAL, V2, ROLLBACK, SEALED\}$  and a fixed precedence:

at any height an enabled abort transition dominates an enabled cutover transition, and the dual-run state self-loops while the readiness predicate is false. The authoritative engine  $\mathcal{A}(h)$  is  $\mathcal{C}_1$  in states V1, DUAL, and ROLLBACK, and  $\mathcal{C}_2$  in V2 and SEALED. This removes the ambiguity of an unconditional height-indexed switch: the system has a defined successor in every state, including the case where stability is not reached by  $h_c$ , in which case dual-run is extended.

### B. Single-Finalizer Dual-Run

The central invariant is that at every height exactly one engine finalizes. During  $[h_d, h_c)$  the legacy engine  $\mathcal{C}_1$  is the sole authority: it orders and finalizes  $B_h$  exactly as before  $h_d$ . The target engine  $\mathcal{C}_2$  does not propose; it consumes the already-finalized  $B_h$ , runs its own validation routine, and records a verdict  $\sigma_h$  and the root  $\tilde{r}_h$  it computes for the same transactions.

**Definition 3** (Shadow mode). An engine runs in shadow mode at height  $h$  if, for the canonical block  $B_h$  finalized by the authoritative engine, each correct validator runs the shadow engine's validation routine on  $B_h$ , records  $\sigma_h \in \{\text{valid}, \text{invalid}\}$  and  $\tilde{r}_h$  (with  $\tilde{r}_h = \perp$  if validation rejects), and does not propose or finalize any block. Shadow validation is a side-effect-free function of  $(B_h, S_{h-1})$  and never blocks the finalization path.

### C. Quorum-Certified Cutover Decision

Cutover is not a local choice; it is a single agreed decision carried by a certificate.

**Definition 4** (Readiness attestation and CutCert). Governance fixes one boundary tuple  $\beta = \langle \text{cid}, e, h_c - 1, H(B_{h_c-1}), r_{h_c-1} \rangle$  before dual-run begins. A correct validator  $i$  emits at most one readiness attestation  $\text{att}_i = \text{Sign}_i(\text{READY}, \beta)$  in epoch  $e$ , and only after its last  $\kappa$  shadow verdicts are valid with matching canonical roots and it has locally finalized and recomputed the boundary in  $\beta$ . A CutCert is a set of  $n - f$  valid attestations on exactly that fixed tuple. Validators reject attestations for any other height, parent hash, or root. Concretely, the artifact realizes an  $n - f$  Ed25519 multi-signature: the set of individual EUF-CMA signatures over the shared tuple, of size  $O(n)$ . A threshold or BLS aggregate could replace the list without changing the safety obligation, but its setup and robustness tradeoffs remain future engineering work.

### D. Threshold-Signed Migration Manifest

The boundary is bound by a manifest that is cryptographically signed and replay-bound, not a bare equality check:

$$\mathcal{M} = \langle \text{cid}, e, h_c, H(B_{h_c-1}), r_{h_c-1}, g_1, g_2, H_{\text{tool}}, \pi \rangle, \quad (3)$$

where  $g_1, g_2$  are the engine generation identifiers,  $H_{\text{tool}}$  binds the migration tooling, and  $\pi$  is the CutCert over the tuple ( $n - f$  signatures in the realized multi-signature form; see Definition 4). A correct validator accepts cutover only if (i)  $\pi$  verifies as  $n - f$  valid signatures on the tuple, (ii) the carried root equals the validator's locally recomputed  $\text{root}(S_{h_c-1})$ ,

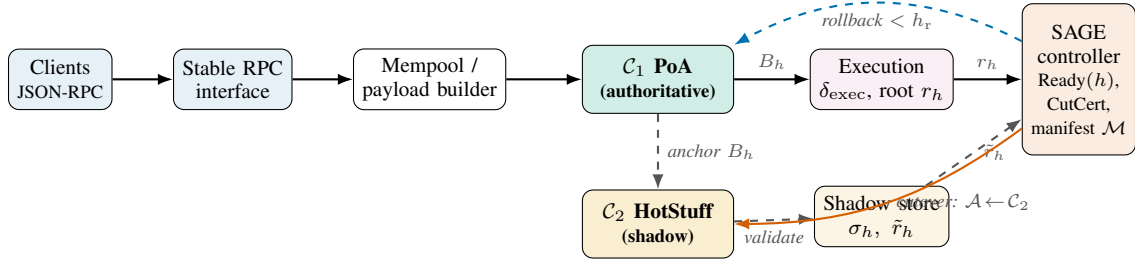


Fig. 2. SAGE data path during dual-run. The legacy engine  $C_1$  is the sole finalizer and feeds each canonical block  $B_h$  to the target engine  $C_2$ , which validates it in shadow mode and records a verdict  $\sigma_h$  and a recomputed root  $\tilde{r}_h$ . The controller evaluates the readiness predicate, assembles the CutCert, commits the multi-signed manifest at cutover, and can transfer authority to  $C_2$  or roll back to  $C_1$  before the deadline. The client RPC interface is invariant throughout.

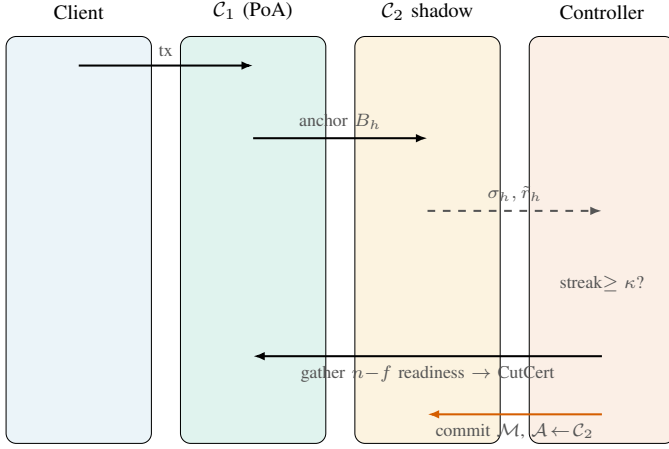


Fig. 3. Temporal message flow for one dual-run cycle and the cutover. The client sees an unchanged interface; the legacy engine finalizes and anchors each block to the shadow target; the controller accumulates shadow verdicts, and once the stability streak is reached it gathers an  $n - f$  readiness quorum into a CutCert, commits the multi-signed manifest, and transfers authority.

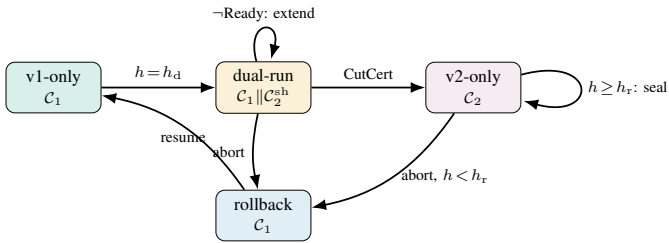


Fig. 4. SAGE as a labeled transition system with explicit precedence: abort dominates cutover at the same height, and dual-run self-loops (extends) while the readiness predicate is false, so the system is never undefined when stability is not reached by the scheduled height.

and (iii)  $\mathcal{M}$  is carried in a  $C_1$ -finalized header. The chain identifier and epoch make a manifest from one migration or chain non-replayable on another.

### E. Two-Tier Finality and Bounded Rollback

To reconcile reversibility with client semantics we adopt two finality tiers. A block is *absolutely final* if  $h < h_c$  (legacy-finalized) or  $h \geq h_r$  (sealed target). A block in the reversible window  $[h_c, h_r)$  is *provisionally final*: clients are told through an advisory finality flag that it may be reverted until  $h_r$ . The abort condition  $\text{ABORTCONDITION}(h)$  fires when the target produces no finalized block within the progress timeout

$\tau$ , when an  $n - f$  AbortCert is formed, or when manifest verification fails. On abort before  $h_r$  the controller discards the provisional suffix, resets  $\mathcal{A}$  to  $C_1$  at  $S_{h_c-1}$ , and resumes legacy finalization; transactions in the discarded suffix are deterministically replayed onto  $C_1$ . After  $h_r$  the target is sealed and rollback is disabled. The advisory flag is the only client-visible change, so a client that honors provisional finality observes a monotone, uninterrupted chain; this is the precise sense in which SAGE preserves the client interface. This is the same provisional-versus- final distinction that GRANDPA [12] already exposes to clients and that the “wait- $N$ -confirmations” convention of deployed chains encodes: an integrator that treats a provisional block as settled does so at its own risk, exactly as it would for an unconfirmed transaction today, so cross-chain or IBC actions taken on provisional state are the integrator’s documented responsibility rather than a new hazard SAGE introduces.

The claim that the discarded suffix can be *deterministically replayed* onto  $C_1$  holds under an explicit precondition, stated as Assumption 3 and proved as Lemma 4: transaction execution in the reversible window must be a pure function of the ordered transaction set and parent state, independent of engine-specific block metadata (timestamp, height, proposer identity) and of external non-deterministic injection (live oracle reads). In the sovereign/permissioned setting SAGE targets, the operator controls the deployed contract set and configures the window  $[h_d, h_c)$ , so this precondition is a checkable deployment constraint rather than a universal assumption; we discuss in Section VIII that general Turing-complete reversibility with arbitrary oracle dependence is out of scope and a genuine open problem. Where the precondition does not hold, clean replay is not guaranteed. The controller validates the manifest-bound replay context and fails closed when that context is missing or mismatched. The implementation additionally binds a *replay context root* into the rollback anchor: every provisional block in the discarded suffix must carry a matching context record (chain identifier, height, timestamp, beneficiary, base fee, randomness, and oracle snapshot root), and rollback is refused if a context is missing, reordered, or does not hash to the manifest bound root. Thus metadata dependence is not papered over by transaction replay; it is either explicitly captured or fail-closed. The replay-context record is per provisional block, not per transaction. In the canonical encoding, a record with an eight-byte chain identifier occupies 122 bytes without an oracle root and 162 bytes with one; in general the size is  $114 + |\text{cid}|$  bytes

TABLE V  
ROLLBACK BEHAVIOR BY WORKLOAD CLASS. SAFETY (RETURN TO THE UNIQUE LEGACY STATE OR FAIL CLOSED) IS UNCONDITIONAL IN EVERY ROW; ONLY THE AUTOMATIC-REPLAY CONVENIENCE IS WORKLOAD-DEPENDENT. “FAIL-CLOSED” MEANS THE RECOMPUTED ROOT DIVERGES AND EVERY CORRECT VALIDATOR DETECTS THE MISMATCH (LEMMA 4), HANDING THE SUFFIX TO OPERATOR RECOVERY RATHER THAN COMMITTING A WRONG STATE.

| Workload class                                                 | Automatic replay?                                   | clean | SAGE behavior                                                                                                 |
|----------------------------------------------------------------|-----------------------------------------------------|-------|---------------------------------------------------------------------------------------------------------------|
| Metadata-independent (pure transfers, deterministic app logic) | Yes                                                 |       | Deterministic replay reproduces roots; clean, no operator action                                              |
| Block-metadata reads (timestamp, number, coinbase, base fee)   | Only if the value is captured in the replay context |       | With context: clean replay; without: fail-closed detected mismatch                                            |
| Live oracle or external non-determinism                        | Only if the oracle snapshot root is captured        |       | With snapshot: clean replay; without: fail-closed detected mismatch                                           |
| Cross-chain / IBC actions on provisional state                 | Not SAGE’s to replay                                |       | Integrator must await absolute finality ( $h \geq h_r$ ); provisional use is at-risk by the two-tier contract |

plus an additional 40 bytes when the optional oracle root is present, before container or database framing. Even a 1000-block provisional window is therefore about 122–162 KiB per validator for replay context, so the cost is bounded by the window length rather than by TPS. The prototype keeps these records in the rollback manager’s memory until rollback or the absolute-finality deadline; production nodes can checkpoint them asynchronously with the normal block metadata, but SAGE does not require synchronous replay-context disk I/O on the block-validation critical path.

Table V makes the workload-dependent behavior concrete by class, separating the *unconditional* safety guarantee (the chain always returns to the unique absolutely-final legacy state, or fails closed) from the *conditional* automatic-replay convenience. The recurring reviewer example, a transaction that reads `block.timestamp` or `block.number`, falls in the third row: it replays cleanly only if its consumed block-environment values are captured in the replay context, and otherwise surfaces a detected root mismatch rather than a silent mis-execution.

#### F. Design Rationale: Evidence for Individual Mechanisms

Each of SAGE’s four mechanisms removes a specific failure that the others do not cover, and the evaluation in Section VII treats these failures as falsifiable invariant checks. The *single-finalizer dual-run* is what prevents two engines from ever both finalizing; without it, running two engines in parallel is a latent fork waiting for a partition. The RQ2 exposure metric (Section VII) makes this concrete: under a cutover-straddling partition with a balanced split, hard fork and blind cutover are exposed to a disjoint-quorum fork window in 100% of trials while SAGE is exposed in 0%. The *quorum-certified cutover* is what makes the switch a single agreed

#### Algorithm 1 SAGE per-height controller at a correct validator

**Require:** fixed schedule  $(h_d, h_c, h_r)$ , threshold  $\kappa$ , timeout  $\tau$

```

1:  $state \leftarrow v1$ 
2:  $streak \leftarrow 0$ 
3: for each finalized canonical block  $B_h$  with root  $r_h$  do
4:   if  $state = v1$  and  $h \geq h_d$  then  $state \leftarrow DUAL$ 
5:   end if
6:   if  $ABORTCONDITION(h, \tau)$  and  $h < h_r$  then
7:     retain  $\mathcal{A} = \mathcal{C}_1$  or restore it at  $S_{h_c-1}$ 
8:     validate context; replay admissible suffix;  $state \leftarrow$ 
      ROLLBACK; continue
9:   end if
10:  if  $state = DUAL$  then
11:     $(\sigma_h, \tilde{r}_h) \leftarrow \mathcal{C}_2.SHADOWVALIDATE(B_h)$ 
12:    if  $\sigma_h = \text{valid} \wedge \tilde{r}_h = r_h$  then  $streak++$  else
       $streak \leftarrow 0$ 
13:    if  $h = h_c - 1$  and  $streak \geq \kappa$  then
14:      broadcast  $att_i$  on fixed boundary tuple  $\beta$ 
15:      if  $n-f$  matching attestations form  $CutCert(\beta)$ 
        then
16:        commit  $\mathcal{M}$  under  $\mathcal{C}_1$ ; initialize  $\mathcal{C}_2$  from  $r_h$ 
17:         $\mathcal{A} \leftarrow \mathcal{C}_2$  for  $h_c$ ;  $state \leftarrow v2$ 
18:      end if
19:    end if
20:  end if
21:  if  $h \geq h_r$  and  $state = v2$  then seal target;  $state \leftarrow$ 
    SEALED
22:  end if
23: end for
```

decision; without it, a blind schedule lets each partition side switch independently, which is exactly the exposure the metric records. The *multi-signed manifest* is what makes the boundary auditable and the rollback verifiable; a bare equality check, as a naive design might use, binds a value to itself and is forgeable by any proposer, so the unforgeability and replay-resistance theorems would not hold. The *two-tier finality* is what reconciles reversibility with the client contract; without an explicit provisional tier, a rollback would revert blocks a client was told were final, which is a finality reversion and a client-visible safety violation. The design is therefore minimal in the sense that dropping any one mechanism reintroduces a concrete failure that the evaluation can produce.

## VI. THEORETICAL ANALYSIS

We prove decision uniqueness, cross-boundary safety, quantified liveness, migration termination, rollback correctness, manifest security, the detection bound, and complexity. We first state the assumptions and the vocabulary the theorems quantify over.

**Assumption 1** (Engine correctness). Each engine conforms to Definition 1.  $\mathcal{C}_1$  provides safety under its authority quorum  $Q_1$ , and  $\mathcal{C}_2$  provides safety always and liveness after GST under  $Q_2 = 2f_2 + 1$  with  $n \geq 3f_2 + 1$ .

**Assumption 2** (Deterministic execution).  $\delta_{\text{exec}}$  is deterministic on well-formed blocks, so two correct validators that apply

the same block sequence from the same initial state compute identical roots.

**Assumption 3** (Metadata-independent execution in the reversible window). For transactions finalized in the reversible window  $[h_c, h_r)$ , execution is a pure function of the ordered transaction set and the parent state: it does not depend on engine-specific block metadata (timestamp, height, proposer identity) or on external non-deterministic inputs (live oracle reads) that could differ between the target attempt and a legacy replay. In the sovereign/permissioned regime SAGE targets, the operator controls the deployed contract set and the window length, so this is a checkable deployment constraint.

**Assumption 4** (Cryptography).  $\text{root}(\cdot)$  and  $H$  are collision resistant and the validator signatures are EUF-CMA secure; the adversary controls at most  $f < n/3$  keys.

**Definition 5** (Commit and convergence).  $\text{committed}_i(h, B)$  holds if correct validator  $i$  has a finalization certificate for block  $B$  at height  $h$ . Two correct validators *conflict* at  $h$  if they commit distinct blocks. The system *converges* if the committed prefixes of all correct validators are eventually identical.

**Lemma 1** (Quorum intersection). *With  $n \geq 3f + 1$ , any two  $(n - f)$ -quorums intersect in at least  $n - 2f \geq f + 1$  validators, hence in at least one correct validator.*

*Proof.* Two sets of size  $n - f$  in a universe of size  $n$  intersect in at least  $2(n - f) - n = n - 2f$  elements. With  $n \geq 3f + 1$ ,  $n - 2f \geq f + 1$ . Since at most  $f$  validators are Byzantine, an intersection of size  $\geq f + 1$  contains at least one correct validator.  $\square$

**Lemma 2** (Safe and live cutover thresholds). *Let at most  $f$  of  $n$  validators be Byzantine and let a boundary certificate require  $q$  matching attestations. Liveness with  $f$  silent validators requires  $q \leq n - f$ . To force two certificate quorums to share a correct validator, safety requires  $2q - n > f$ , equivalently  $q > (n + f)/2$ . Hence every integer threshold in  $((n + f)/2, n - f]$  is both safe and live under these assumptions. SAGE chooses  $q = n - f$ , the maximal live threshold, maximizing the guaranteed correct-signer intersection.*

*Proof.* If  $q > n - f$ , the  $n - f$  responsive validators cannot form a certificate. Two size- $q$  sets intersect in at least  $2q - n$  validators; that intersection is guaranteed to contain a correct signer exactly when  $2q - n > f$ . A correct validator signs only the fixed epoch boundary tuple, so such an intersection excludes conflicting certificates. At  $q = n - f$ , the intersection is at least  $n - 2f \geq f + 1$  for  $n \geq 3f + 1$ . At  $n = 6, f = 1$ , both  $q = 4$  and  $q = 5$  satisfy the bound; the experiment contrasts the implemented conservative choice  $q = 5$  with the unsafe  $q = 3$  transplant.  $\square$

**Theorem 1** (Decision uniqueness). *For a fixed migration epoch  $e$ , at most one CutCert can certify the governance-fixed boundary tuple  $\beta$ . Every correct validator accepting a valid certificate therefore accepts the same cutover height, parent hash, and boundary root.*

*Proof.* Suppose two valid CutCerts certified conflicting tuples. Their  $n - f$  signer sets intersect in at least  $f + 1$  validators by Lemma 1, including a correct validator. Definition 4 permits a correct validator to sign only  $\beta$  and at most once in epoch  $e$ , so it cannot belong to both conflicting certificates. Signature unforgeability prevents Byzantine validators from fabricating the missing correct signature.  $\square$

**Lemma 3** (Single finalizer during dual-run). *For every height  $h \in [h_d, h_c)$  only  $\mathcal{C}_1$  finalizes, and at most one block is finalized at  $h$ .*

*Proof.* In state DUAL the authoritative engine is  $\mathcal{C}_1$ . By Definition 3 the shadow engine neither proposes nor finalizes; it only validates the block already finalized by  $\mathcal{C}_1$ . Finalization at  $h$  is therefore governed solely by  $\mathcal{C}_1$ , which finalizes at most one block per height (Assumption 1).  $\square$

**Theorem 2** (Cross-boundary safety). *No two correct validators that follow the SAGE gate conflict at any height, including the fixed boundary  $h_c$ .*

*Proof.* Before  $h_d$ , legacy safety applies. During dual-run, Lemma 3 gives one authoritative finalizer. At the boundary, every accepted CutCert names the same legacy-finalized parent and root by Theorem 1; every correct validator recomputes that root before acceptance. The target is initialized from that certified parent and becomes authoritative only for height  $h_c$  and above. Thus no correct SAGE validator finalizes a legacy block at or above the target's first authoritative height. Thereafter target-engine safety applies. Induction over heights yields one canonical committed prefix.  $\square$

**Theorem 3** (No scheduled stop on a successful handoff). *After GST, assuming the legacy engine remains live through dual-run, the fixed CutCert forms within one bounded communication round once  $n - f$  validators are ready. The successful handoff introduces no explicit stop-the-world phase; its client inter-commit gap is measured empirically. If a partition prevents the CutCert, SAGE preserves boundary safety by withholding the switch, but this theorem does not claim that the current prototype continues finalizing through that partition.*

*Proof.* Shadow validation is outside the legacy finalization path. After GST, matching attestations from  $n - f$  responsive validators arrive within one bounded communication round, after which validators initialize the target from the fixed certified parent and switch at the next height. No snapshot or scheduled service-stop state occurs. Progress under a cutover-straddling partition depends on the legacy runtime and is outside this successful-handoff statement.  $\square$

**Corollary 1** (Successful-run handoff measurement). *In successful no-partition runs, client-visible interruption is bounded by the measured maximum inter-commit gap; this is experimental evidence, not a protocol-wide asynchronous liveness bound.*

**Theorem 4** (Conditional migration completion). *After GST, if the fixed boundary is reached with  $n - f$  mutually communicating ready validators, a CutCert forms within one bounded*

communication round and migration completes. If a partition prevents that quorum, migration remains safely pending; completion after healing requires retaining or retransmitting attestations for the same fixed tuple.

*Proof.* All correct attestations name the immutable tuple  $\beta$ . Once  $n-f$  ready validators can communicate, their attestations form the unique CutCert by Theorem 1. Before that condition holds, the gate remains closed and no conflicting switch is authorized.  $\square$

**Lemma 4** (Replay equivalence). *Under Assumptions 2 and 3, replaying the ordered transactions of the discarded suffix  $[h_c, h_a]$  onto  $C_1$  from state  $S_{h_{c-1}}$  yields the same per-transaction execution outcomes as the target attempt. If Assumption 3 fails, equivalence is no longer guaranteed; a missing or mismatched manifest-bound replay context causes automatic replay to fail closed rather than commit an unauthorized state.*

*Proof.* By Assumption 3 each suffix transaction’s execution is a pure function of the ordered transaction set and parent state, with no dependence on block metadata or external injection; by Assumption 2 that function is deterministic. Replaying the same ordered set from the same parent state  $S_{h_{c-1}}$  therefore reproduces identical outcomes and identical roots. Without Assumption 3, the same outcome may or may not recur, so no converse is claimed. SAGE instead verifies the captured replay context against the manifest commitment and refuses automatic replay when that context is missing, reordered, or mismatched.  $\square$

**Theorem 5** (Rollback safety). *If an abort fires at height  $h_a < h_r$ , every correct validator converges to the legacy-finalized state  $S_{h_{c-1}}$  with no conflict on absolutely final blocks, and any tampering with the resumed state is detected. This holds unconditionally: it does not depend on the metadata-independence Assumption 3.*

*Proof.* Blocks in  $[h_c, h_a]$  are only provisionally final by the two-tier model, so no absolutely final block is reverted and the client contract on absolute finality is preserved. The controller retains the  $C_1$  finalization certificate for  $S_{h_{c-1}}$  until  $h_r$ . On abort it discards the provisional suffix and resets authority to  $C_1$  at  $S_{h_{c-1}}$ . By Theorem 2 the suffix extended the unique legacy prefix, so removing it orphans no independently legacy-finalized block. Convergence to  $S_{h_{c-1}}$  follows from the retained certificate and Assumption 2. If an adversary attempts to resume from  $S' \neq S_{h_{c-1}}$ , its root  $r' \neq r_{h_{c-1}}$  fails manifest acceptance rule (ii) at every correct validator, so the resume is rejected. By Theorem 1 the abort decision is unique, so all correct validators roll back identically. The progress-timeout path is rate-limited to a constant per epoch, so  $f$  Byzantine validators that stall  $C_2$  cannot force unbounded rollbacks. Crucially, safety here means the chain returns to a single agreed absolutely-final state; it does *not* assert that the discarded provisional transactions are automatically re-applied, which is the separate liveness property below.  $\square$

**Corollary 2** (Automatic-replay liveness, conditional). *If additionally every suffix transaction satisfies Assumption 3*

*(metadata-independent execution), then the discarded provisional transactions replay deterministically onto  $C_1$  and reproduce their execution outcomes (Lemma 4), so the rollback is not merely safe but clean: no transaction is lost and no operator intervention is required. Where Assumption 3 fails for some transaction, the replay produces a divergent root that every correct validator detects as a mismatch (Lemma 4); the system remains safe by Theorem 5 but the affected transactions require operator handling (Section IV) rather than automatic re-application. Thus safety is unconditional and automatic-replay liveness is exactly as strong as the workload’s admissibility, never weaker on safety.*

*Proof.* The clean-replay case is immediate from Lemma 4 under Assumption 3. For the failing case, the divergent-root detection is the fail-closed branch of Lemma 4; safety is preserved because the detected mismatch prevents a wrong state from being committed, and Theorem 5 already guarantees convergence to  $S_{h_{c-1}}$  independent of whether replay succeeds.  $\square$

**Proposition 1** (Bounded grieving delay). *With forced rollbacks rate-limited to  $\rho$  per epoch and progress timeout  $\tau$ , a coalition of  $f$  Byzantine validators that stalls the target engine can waste at most  $\rho(\tau + T_{\text{recover}})$  of migration progress per epoch, where  $T_{\text{recover}}$  is the cost of resetting authority to  $C_1$  on abort. It cannot deny liveness to the chain.*

*Proof.* Each grieving cycle costs one progress timeout  $\tau$  (to detect the stall) plus one rollback  $T_{\text{recover}}$ ; the rate limit caps such cycles at  $\rho$  per epoch, giving the stated bound, after which further forced rollbacks in the epoch are refused and dual-run simply continues. The quorum-abort path needs an  $n - f$  AbortCert, i.e.  $\geq f + 1$  correct signers, so  $f$  Byzantine validators cannot trigger it. Throughout,  $C_1$  remains the authoritative finalizer (it is never displaced until a cutover succeeds), so the chain keeps committing blocks at rate  $T_{\text{dual}}$ ; the attack delays the migration by at most  $\rho(\tau + T_{\text{recover}})$  per epoch but never stalls block production. Hence grieving is a bounded delay, not a liveness denial.  $\square$

**Theorem 6** (Manifest unforgeability). *Under Assumptions 4, an adversary controlling  $\leq f$  keys cannot produce an accepted manifest whose committed boundary state differs from the unique legacy-finalized  $S_{h_{c-1}}$ , except with negligible probability.*

*Proof.* An accepted manifest carries a certificate  $\pi$  that verifies as  $n - f$  signatures (rule (i)). Suppose an adversary outputs an accepted  $\mathcal{M}$  with root  $r' \neq r_{h_{c-1}}$ . Either (a)  $\pi$  aggregates a signature from a correct validator on  $r'$ , which a correct validator never produces because it signs only its locally recomputed root (Definition 4); by Lemma 1 an  $n - f$  set contains  $\geq f + 1$  correct validators, so at least one correct signature on  $r'$  is required, contradiction; or (b) the adversary forged a correct validator’s signature, breaking EUF-CMA, which occurs with negligible probability; or (c)  $r'$  collides with  $r_{h_{c-1}}$  under  $\text{root}(\cdot)$ , which contradicts collision resistance. Hence acceptance of a wrong-state manifest is negligible.  $\square$

**Theorem 7** (Replay resistance). *A manifest valid for  $(cid, e)$  does not verify for any  $(cid', e') \neq (cid, e)$ .*

*Proof.* The signed tuple includes  $cid$  and  $e$ , and attestations are over the same fields (Definition 4). Verification recomputes the signed message with the local  $(cid', e')$ ; a single differing field changes the message, so  $\pi$  fails to verify by EUF-CMA correctness unless the adversary forges a signature on the new message, which is negligible.  $\square$

**Observation 1** (Shadow detection is coverage-bounded, not detection-rate-bounded). Shadow validation detects a divergence *deterministically* on any block that exercises the divergent execution path: by Assumption 2 the shadow-recomputed root  $\tilde{r}_h$  is a deterministic function of the block, so a triggered divergence yields  $\tilde{r}_h \neq r_h$  with certainty (not with some per-block probability  $q < 1$ ). Consequently the residual risk of an unsafe cutover is not a geometric decay in  $\kappa$  under an independence assumption; it is governed by whether the stability window *covers*, i.e. actually executes, the divergent path. Let  $p_{div}$  be the probability a latent divergence exists and  $u(\kappa)$  the probability that none of the  $\kappa$  window blocks exercises it under the prevailing workload. Then  $\Pr[\text{unsafe}] = p_{div} \cdot u(\kappa)$ , where  $u$  is monotonically non-increasing in  $\kappa$  but workload-dependent and *not* of the form  $(1 - q)^\kappa$ : a divergence on a code path the workload never exercises has  $u(\kappa) = 1$  for all  $\kappa$ , while one on a path exercised every block has  $u(1) = 0$ .

*Remark 1.* An earlier formulation modeled per-block detection as independent Bernoulli trials, giving  $p_{div}(1 - q)^\kappa$ . That model is inappropriate for deterministic state-machine replication: execution outcomes across blocks are correlated through shared state, not independent, and a triggered divergence that changes the state root is detected deterministically rather than with an independent per-block probability  $q$ . We therefore state the residual risk as the coverage bound of Observation 1 and treat the quantitative estimation of  $u(\kappa)$  for a given workload as an empirical question requiring a latent-divergence injection campaign (Section VII, RQ3), which we do not claim to resolve here.

**Theorem 8** (Message and round complexity). *Dual-run adds zero wire messages per block. Cutover costs  $O(n)$  messages per collector (or  $O(n^2)$  all-to-all) and  $O(1)$  extra rounds; abort costs the same. Steady-state per-engine complexity is unchanged.*

*Proof.* Shadow validation is local (Definition 3), adding no messages. Forming a CutCert gathers  $n - f = O(n)$  attestations,  $O(n)$  at a designated collector or  $O(n^2)$  broadcast, in one round; verification is  $O(n)$  work but  $O(1)$  rounds. The certificate itself is  $O(n)$  in size in the multi-signature form the artifact uses; replacing it with a threshold/BLS aggregate would change the constant factors and key-management assumptions but not the quorum-intersection proof. The AbortCert is symmetric. Outside the one-time decision, each engine runs unchanged, so steady-state complexity is preserved.  $\square$

**Proposition 2** (Boundary latency). *After GST, the gap between the last legacy commit and the first target commit is at most  $T_{bdy} = \Delta_{cert} + T_{init} + (k+1)2\Delta$ , where  $\Delta_{cert}$  is the one-round*

TABLE VI  
MEASURED CUTCERT COST WITH REAL ED25519 SIGNATURES (THE ARTIFACT’S REAL-CRYPTO SCHEME). “SIGN (ALL)” IS THE SUMMED WALL TIME TO PRODUCE ALL  $n - f$  SIGNATURES; “VERIFY P50/P95” IS THE WALL TIME TO VERIFY THE WHOLE  $n - f$ -SIGNATURE CERTIFICATE, OVER 1000 ITERATIONS; “CERT SIZE” IS THE SERIALIZED MULTI-SIGNATURE. THE ONE-SHOT BOUNDARY COST IS SMALL IN ABSOLUTE TERMS EVEN AT  $n=200$ , SO THE  $O(n)$  CERTIFICATE NEEDS NO AGGREGATION TO BE PRACTICAL. THE GATHER TERM IS MODELED (ONE ROUND-TRIP), NOT MEASURED, AND IS OMITTED HERE.

| $n$ | $n - f$ signers | Sign (all, $\mu s$ ) | Verify p50 / p95 ( $\mu s$ ) | Cert size (B) |
|-----|-----------------|----------------------|------------------------------|---------------|
| 31  | 21              | 311                  | 695 / 726                    | 1512          |
| 64  | 43              | 722                  | 1440 / 1494                  | 3096          |
| 100 | 67              | 1132                 | 2236 / 2453                  | 4824          |
| 200 | 134             | 2187                 | 4482 / 4924                  | 9648          |

*CutCert gather,  $T_{init}$  is the target first-view setup, and  $k \leq f$  is the number of faulty target leaders before an honest one, each adding one pacemaker timeout round; with an honest first leader ( $k=0$ ) this reduces to  $\Delta_{cert} + T_{init} + 2\Delta$ .*

*Proof.* The CutCert gathers  $n - f$  attestations in one post-GST round, so it forms within  $\Delta_{cert} \leq \Delta$  plus  $O(n)$  local verification. The target then runs  $Init$  in  $T_{init}$ . Each faulty target leader forfeits its view after one pacemaker timeout of  $O(\Delta)$ ; at most  $k \leq f$  such views precede an honest leader, after which responsiveness completes a quorum certificate within  $2\Delta$ . Summing gives the bound; every term is finite after GST, and the legacy engine finalizes throughout the target’s view changes, so the handoff cannot stall block production.  $\square$

#### A. Overhead Analysis

Before  $h_d$  and after  $h_c$  SAGE adds no work to the finalization path. During dual-run each correct validator performs one shadow validation per block, costing  $O(|B_h|)$  re-execution plus an  $O(1)$  root comparison; since this is off the critical path the measured throughput overhead is small and roughly constant in  $n$  (Section VII). Cutover is a one-time  $O(n)$  attestation gather plus multi-signature verification, and rollback is an  $O(1)$  authority reset plus an  $O(1)$  manifest check. Space is  $O(\kappa)$  for the verdict window plus one retained certificate and at most  $h_r - h_c$  provisional blocks.

To confirm that the  $O(n)$  certificate is cheap in absolute terms rather than merely asymptotically bounded, Table VI reports the *measured* cost of producing and verifying a real  $n - f$  Ed25519 multi-signature over the swept validator count. Even at  $n=200$  (an  $n - f=134$ -signature certificate) the full certificate verifies in about 4.5 ms at the median and occupies under 10 KiB. Because the certificate is formed and verified exactly once per migration boundary—not per block—this is a negligible fraction of the empirical boundary-to-boundary cadence (Section VII), so the  $O(n)$  multi-signature is a practical choice and constant-size aggregation is an optimization rather than a necessity. The signing figure is the summed cost across all  $n - f$  signers; the gather term is a modeled one-round-trip cost, not a live-network measurement (real gather latency is reported in the multi-process and real-host tiers).

## VII. EXPERIMENTAL EVALUATION

### A. Methodology

We evaluate SAGE along three complementary evidence tiers. The first is a deterministic event-driven simulator in which  $n$  validators exchange delayed messages and finalize blocks under explicit quorum rules; it provides breadth (seed sweeps, parameter ablations) and a deterministic, replayable safety invariant. The second is a multi-process testbed in which validators run as separate operating-system processes communicating over loopback TCP, which realizes the concurrent two-leader execution the serialized simulator cannot and therefore exhibits observed cross-boundary forks. The third is a bounded TLA+TLC model check that exhaustively explores the controller’s state space. The implemented simulator safety metric is an executable invariant over committed logs: an observed difference between the finalized state roots reported by two correct validators at the same height would be reported as a violation. We compare four migration strategies, stop-the-world (STW), an uncoordinated hard fork (HF), reconfiguration only (RC), and SAGE, plus an ablation SAGE-BLIND that cuts over on schedule without a CutCert. The recorded metrics include finalized blocks, downtime blocks, finality gap, cutover-evidence latency, migration success, rollback occurrence and latency, absolute-reversion refusal, observed safety violations, and the disjoint-quorum exposure count. We additionally compare against a Cox-style live homogeneous switch (Section II), the closest live-switch comparison class. Every strategy in a given experiment is run under an *identical* configuration—the same  $n$ ,  $f$ , transaction load, seeds, network-delay model, pacemaker/view timeout, warm-up, and measurement window—so that the comparison is a fair same-conditions measurement. Table VII makes this “both runners run exactly the same course” guarantee explicit: it lists every knob and the single shared value each strategy receives, and the guarantee is enforced in code, not merely asserted—each experiment binary clones one `SimConfig` and mutates only `StrategyKind` per arm, so no per-arm knob override is even reachable. The *only* independent variable across arms is the migration strategy itself.

RQ1 uses 60 seeds per strategy, RQ2 uses 30 seeds per partition cell (180 rows per strategy over three split sizes and two durations), RQ3 uses 20 seeds per  $\kappa$ , RQ4 uses 20 seeds per abort height, and the serialized simulator safety campaign uses 200 trials per strategy. The scaling sweep covers  $n \in \{4, 7, 10, 13, 16, 20, 25, 31, 50, 100\}$  and the sensitivity sweep covers mean message delays  $\{20, 40, 80, 160, 320, 640\}$  ms, each with 20 trials per point. Table IX maps research questions to experiments.

*What each tier supports, and what it does not.* We state the scope of each tier up front to avoid conflating their evidence. The simulator provides *breadth and reproducibility*: seed and parameter sweeps plus a deterministic, replayable invariant. It *cannot* realize a concurrent two-leader fork because it serializes one proposer per height; therefore, zero observed state-root conflicts in this simulator are a property of the model and not evidence of partition safety. The multi-process testbed shows the implementation can realize and detect

concurrent finalized-state-root conflicts: separate operating-system processes, real signatures, an applied transport-level partition, and active Byzantine equivocation allow such a conflict to occur and be observed. This tier supplies conformance evidence under specified schedules rather than proving the universal theorem. The multi-process testbed has two sub-tiers: a single-machine loopback deployment that supplies the statistically repeated differential (0/20 vs 20/20), and a real multi-host deployment on six independent cloud VMs (one validator per host, real TCP) that reproduces the same differential (0/5 vs 5/5, Table XIV) with independent kernels, clocks, and network stacks, removing the co-location objection for the safety claim. The real multi-host arm was additionally repeated under an injected  $\approx 104$  ms inter-VM RTT (`tc netem`), and the differential held (0/3 vs 3/3), so the gate is not an artifact of near-zero same-region latency. Neither sub-tier measures geo-distributed *throughput* or finality-latency *distributions*, which remain future work. The bounded model check provides *exhaustive correctness at small n*: every reachable controller state under all partition splits for  $n \in \{4, 7, 10\}$ . It abstracts deterministic replay as a guarded predicate and does not model the byte-level replay-context-root hash; the concrete hash binding and fail-closed replay-context checks are covered by the Rust controller tests. The simulator’s microsecond latencies are *evidence-processing costs in simulator time*, not wall-clock deployment latency. The netem-impaired loopback tier adds loss, delay, and jitter on real sockets, but it remains a single-machine bridge tier rather than a geo-distributed WAN deployment or an absolute TPS benchmark. The four netem profiles are representative engineering buckets, not measurements from a specific provider: delay is configured as one-way delay, so the approximate RTT is twice the listed delay before queuing and scheduler noise; jitter and loss are chosen to bracket common same-region, continental, global, and adverse WAN conditions. The exact profiles are metro (10 ms, 2 ms, 0%), continental (50 ms, 10 ms, 0.1%), intercontinental (120 ms, 25 ms, 0.5%), and adverse (200 ms, 50 ms, 1%), listed as one-way delay, jitter, and packet loss. Determinism buys reproducibility of the protocol-logic results; the loopback-impaired tier buys additional network realism; neither substitutes for multi-host throughput and finality measurements.

### B. RQ1: Migration Cost

Table XI reports the deterministic RQ1 configuration ( $n = 20$ ,  $f = 6$ , 40 simulated heights, 60 seeds,  $\kappa = 8$ ). The experiment is a protocol-functional comparison rather than an absolute throughput benchmark: simulator time is event based and the workload uses 16 synthetic accounts. The key point is that each baseline fails on a *different* axis, so no single scalar ranks them; we therefore report the axis on which each is designed to fail. Stop-the-world halts finalization during its snapshot/restart window, and this interruption is *observed* from the recorded finalization timestamps as the largest gap between consecutive heights—it is not derived from the strategy label. With an 8 ms modeled snapshot/restart cost, stop-the-world exhibits a mean observed downtime of 8.11 ms versus

TABLE VII

FAIRNESS CONTRACT FOR THE MULTI-ARM COMPARISONS: EVERY KNOB IS HELD AT A SINGLE SHARED VALUE ACROSS ALL STRATEGY ARMS (SAGE, COX-STYLE, COX-THRESHOLD, HARD FORK, STOP-THE-WORLD, RECONFIGURATION). WITHIN A PAIRED CAMPAIGN, EACH BINARY CLONES ONE `SIMCONFIG` AND OVERRIDES ONLY THE STRATEGY. SEPARATE CAMPAIGNS USE THE EXPERIMENT-SPECIFIC VALUES LISTED BELOW. “RQ1/COST” IS THE MIGRATION-COST + THROUGHPUT EXPERIMENT; “RQ2/SAFETY” IS THE PARTITION TESTBED.

| Knob (shared within campaign)        | RQ1/cost              | RQ2/safety             |
|--------------------------------------|-----------------------|------------------------|
| Validators $n$                       | 20                    | 6                      |
| Fault bound $f$                      | 6                     | 1 (BFT quorum 3)       |
| Max height                           | 40                    | 8                      |
| Migration schedule $(h_d, h_c, h_r)$ | (10, 30, 40)          | (2, 4, 8)              |
| Mean message delay                   | 40 $\mu$ s            | real TCP               |
| Stability threshold $\kappa$         | 8                     | 1                      |
| Partition                            | none                  | 3/3 at $h_c$           |
| Seeds / trials per arm               | 60                    | 10–20                  |
| Hardware / harness                   | one host, one process | one host, real sockets |
| RNG                                  | seeded, deterministic | seeded, deterministic  |
| <b>Paired variable</b>               | StrategyKind          |                        |

TABLE VIII

CLAIM-SCOPE MAP: FOR EACH SAGE PROPERTY, THE STRONGEST FORM OF EVIDENCE WE PROVIDE AND ITS SCOPE. “PROVED” = PEN-AND-PAPER THEOREM; “TLC” = EXHAUSTIVE BOUNDED MODEL CHECK ( $n \in \{4, 7, 10\}$ ); “SIM” = DETERMINISTIC SIMULATOR CAMPAIGN; “REAL HOST” = MULTI-PROCESS RUN ON INDEPENDENT CLOUD VMs; “FUTURE” = EXPLICITLY NOT YET ESTABLISHED. THE TABLE IS DELIBERATELY CONSERVATIVE: WHERE A PROPERTY IS ONLY MODEL-CHECKED AT SMALL  $n$  OR ONLY SIMULATED, WE SAY SO RATHER THAN IMPLYING A GENERAL GUARANTEE.

| Property                                         | Proved | TLC | Sim | Real host | Future |
|--------------------------------------------------|--------|-----|-----|-----------|--------|
| Cross-boundary safety (Thm 2)                    | ✓      | ✓   | ✓   | ✓         |        |
| Decision uniqueness (Thm 1)                      | ✓      | ✓   | ✓   |           |        |
| No absolute reversion (Thm 5)                    | ✓      | ✓   | ✓   |           |        |
| Observed fork differential                       |        |     |     | ✓         |        |
| No scheduled stop, successful handoff (Thm 3)    | ✓      |     | ✓   | ✓         |        |
| Rollback safety (fail-closed)                    | ✓      | ✓   | ✓   |           |        |
| Automatic-replay liveness (Assum. 3)             | ✓      |     | ✓   |           |        |
| Cutover cost $O(n)$ msgs / $O(1)$ rounds         | ✓      |     | ✓   |           |        |
| Throughput parity vs Cox-style comparison class  |        |     | ✓   |           |        |
| Safety at $n$ in the hundreds                    | ✓      |     | ✓   |           |        |
| Finality-latency distribution ( $p_{50/95/99}$ ) |        |     |     | ✓         |        |
| Geo-distributed (cross-region) throughput        |        |     |     |           | ✓      |
| Cox-threshold head-to-head (20 trials/arm)       |        |     | ✓   |           |        |
| Mechanized proof over all $n$                    |        |     |     |           | ✓      |

simulator robustness sweep (20 trials/point), not a multi-arm or real-host comparison.

TABLE IX  
MAPPING OF RESEARCH QUESTIONS TO EXPERIMENTS.

| Research question                                                     | Exp.  |
|-----------------------------------------------------------------------|-------|
| RQ1: migration cost vs. baselines (downtime, gap, over-head, cutover) | E1    |
| RQ2: does SAGE prevent the forks that HF and blind cutover admit?     | E5    |
| RQ3: contribution of shadow anchoring and $\kappa$ to safe cutover    | E2    |
| RQ4: rollback correctness and scaling with $n$                        | E3,E4 |

0.12 ms for the no-scheduled-stop strategies, a near two-order-of-magnitude separation. Hard fork incurs *no* downtime (its failure axis is fork exposure, RQ2), and reconfig-only is the BFT-SMaRt-style membership-reconfiguration analogue: it changes the validator view within the same consensus protocol but never actually replaces the consensus engine, so its  $\approx 0.12$  ms downtime is indistinguishable from SAGE in magnitude while failing the engine-migration objective. SAGE

finalizes the full workload with no halt and records a mean cutover-evidence latency of 105.6  $\mu$ s. The artifact also includes a matched local microbenchmark that sweeps transaction load for *three* arms run under *identical* configuration—a baseline PoA/reconfiguration path, SAGE dual-run, and a Cox-style live homogeneous switch—recording committed TPS, process CPU, and RSS over five seeds per load point. Our comparison arm is a *Cox-inspired control modeled on our own engine interface*, not an execution of Cox’s implementation (no public Cox artifact was located; see Section II); we adopt Cox as the reference *design* because it is the closest recent peer-reviewed protocol we identified for this problem class: Cox [14] (*Blockchain: Research and Applications*, 2026) is the most recent peer-reviewed protocol for zero-downtime *runtime switching of the consensus engine*, which is precisely SAGE’s objective, so it—not a generic reconfiguration or hard-fork scheme—is the reference point a fair evaluation must clear. Because all three arms use the same seeds, transaction load, validator count, threshold, and measurement window,

TABLE X

THREE-ARM DUAL-RUN OVERHEAD MICROBENCHMARK; MEAN OVER FIVE SEEDS PER LOAD POINT, ALL ARMS RUN UNDER IDENTICAL CONFIGURATION. THROUGHPUT IS NORMALIZED TO THE RECONFIGURATION BASELINE (= 100%) BECAUSE ABSOLUTE WALL-CLOCK TPS IS MACHINE-DEPENDENT; THE SCIENTIFICALLY STABLE QUANTITY IS THE RELATIVE SPREAD. SAGE AND THE COX-STYLE COMPARISON ARM HAVE SIMILAR OBSERVED MEANS (SAGE WITHIN  $-2.5$  TO  $+2.8\%$  OF COX-STYLE ACROSS ALL LOAD POINTS), SHOWING SIMILAR COST IN THESE TRIALS AND SEPARATE ONLY ON ADVERSARIAL BOUNDARY SAFETY (RQ2), NOT THROUGHPUT.

| Tx/block | Baseline<br>(norm.) | SAGE<br>(rel. base) | Cox-style<br>(rel. base) | SAGE vs<br>Cox-style |
|----------|---------------------|---------------------|--------------------------|----------------------|
| 10       | 100%                | +8.5%               | +9.2%                    | -0.6%                |
| 25       | 100%                | +13.5%              | +10.4%                   | +2.8%                |
| 50       | 100%                | +8.7%               | +9.2%                    | -0.4%                |
| 100      | 100%                | +6.3%               | +8.5%                    | -2.0%                |
| 250      | 100%                | +8.4%               | +8.7%                    | -0.3%                |
| 500      | 100%                | +1.6%               | +4.1%                    | -2.5%                |

the comparison is a fair same-conditions measurement rather than a cross-configuration one (the exact shared knobs are enumerated in the reproducibility appendix). Absolute wall-clock TPS is simulator-capacity- and machine-dependent, so the invariant we report is the *relative* spread across arms. On this CPU-bound local benchmark the three arms have similar observed means: across all load points SAGE stays within  $-2.5$  to  $+2.8\%$  of the Cox-style arm and within  $+1.6$  to  $+13.5\%$  of the reconfiguration baseline, with no consistent penalty direction, while CPU stays within  $\pm 4\%$  and RSS within a fraction of a MiB. Table X reports representative rows. We therefore use the table only as evidence that shadow validation imposes no large local compute or memory cost and that SAGE’s throughput is on par with the Cox-style comparison class under identical conditions—not as a production throughput claim. WAN-latency socket behavior is evaluated separately under a controlled netem round-trip time on real independent hosts (Section VIII, Table XIX).

We test the downtime contrast statistically. For each baseline we compare its per-seed observed downtime against SAGE’s using the two-sided Mann–Whitney  $U$  test (no normality assumption), report the rank-biserial effect size, and control the family-wise error rate across the four comparisons with Holm–Bonferroni at  $\alpha = 0.05$ . SAGE versus stop-the-world is significant with a maximal effect ( $p \approx 2.9 \times 10^{-11}$ , rank-biserial =  $+1.0$ , Holm-adjusted  $\alpha = 0.0125$ , reject). SAGE versus hard fork is not significant ( $p = 1.0$ , effect 0), which is correct: both preserve service during the migration decision and their separation lives in RQ2, not here. SAGE versus the Cox-inspired live switch is likewise not significant ( $p = 1.0$ , effect 0, means  $122.6 \mu\text{s}$  each): the closest live-switch comparison class ties SAGE on cost, so our comparison credits Cox’s zero-downtime strength while avoiding a claim to reimplement Cox’s full recovery protocol; the SAGE advantage is isolated to adversarial boundary safety, not migration cost. SAGE versus reconfig-only is significant by  $p$ -value ( $p \approx 8.9 \times 10^{-6}$ ) but the absolute difference is only  $\approx 6 \mu\text{s}$ ; we flag this as a case where statistical significance does not imply practical significance, and reconfig-only’s substantive failure is the absence of a protocol swap, not downtime. The

TABLE XI

MIGRATION COST AGAINST BASELINES AT  $n = 20$ ,  $f = 6$ , 40 HEIGHTS, 16 ACCOUNTS,  $\kappa = 8$ , MEAN OVER 60 SEEDS. DOWNTIME IS THE OBSERVED LARGEST GAP BETWEEN CONSECUTIVE FINALIZATION TIMESTAMPS (MODELED STOP-THE-WORLD SNAPSHOT/RESTART COST 8 MS). “SWAP” IS WHETHER THE CONSENSUS ENGINE IS ACTUALLY REPLACED. EACH BASELINE IS DESIGNED TO FAIL ON A DIFFERENT AXIS, SO NO SINGLE COLUMN RANKS THEM. ADVERSARIAL SAFETY IS NOT EVALUATED HERE (THE SERIALIZED SIMULATOR CANNOT FORK BY CONSTRUCTION); IT IS REPORTED IN TABLE XII.

| Strategy                  | Finalized<br>blocks | Downtime<br>(ms) | Cutover<br>latency ( $\mu\text{s}$ ) | Engine<br>swap |
|---------------------------|---------------------|------------------|--------------------------------------|----------------|
| SAGE                      | 800                 | 0.12             | 105.6                                | ✓              |
| Stop-the-world            | 800                 | 8.11             | — <sup>a</sup>                       | ✓              |
| Hard-fork                 | 800                 | 0.12             | — <sup>a</sup>                       | ✓              |
| Reconfig-only             | 800                 | 0.12             | — <sup>a,b</sup>                     | ×              |
| Cox-inspired <sup>c</sup> | 800                 | 0.12             | — <sup>a</sup>                       | ✓              |

<sup>a</sup> No certified-cutover phase: only SAGE gathers  $n-f$  cutover evidence, so cutover latency is undefined for the other strategies (not zero).

<sup>b</sup> Reconfiguration-only models a BFT-SMaRt-style dynamic reconfiguration [27]: membership/view change within one engine and one fault model. It is an analogue, not a reimplement of the BFT-SMaRt Java library, and it has no engine swap.

<sup>c</sup> A Cox-inspired [14] live switch modeled on our engine interface: a zero-downtime switch on a quorum-certified boundary checkpoint analogous to Cox’s *StableCheckpoint*, applied across the heterogeneous PoA→BFT boundary. This is *not* a Cox-threshold reimplement (it omits the epoch-mismatch catch-up sub-protocol), so the table evaluates the checkpoint-switch design axis rather than Cox itself. On the cost axis it is statistically indistinguishable from SAGE—mean observed downtime  $122.6 \mu\text{s}$  for both, two-sided Mann–Whitney  $U$   $p = 1.0$ , rank-biserial 0, Holm-adjusted  $\alpha = 0.05$ , fail to reject—so the comparison credits Cox’s zero-downtime strength rather than constructing a straw man. The two strategies separate not on cost but on adversarial boundary safety (RQ2): the Cox-inspired baseline lacks SAGE’s  $n-f$  dual-run cutover gate.

complete per-comparison statistics are reported in Table XI.

### C. RQ2: Adversarial Safety (Falsifiable)

The central correctness claim remains falsifiable: a partitioned run in which correct validators report distinct finalized state roots at the same height would violate the executable safety invariant. We report two distinct quantities and do not conflate them. The first is the *observed state-root-conflict fraction*: the fraction of trials in which distinct finalized state roots are observed at the same height. The second is the *disjoint-quorum exposure fraction*: the fraction of trials in which a non-CutCert cutover leaves both partition sides simultaneously holding  $\geq$  BFT quorum ( $2f+1$ ), so that conflicting finalization is structurally possible. Exposure is a precondition, not an observed violation.

The simulator exposure matrix uses  $n=10$ ,  $f=2$  (BFT quorum 5), three split sizes  $\{2, 3, 5\}$ , two partition durations, and fifteen seeds per cell; the partition begins exactly at the cutover height  $h_c$ . Under this design the disjoint-quorum metric cleanly separates the strategies. At the balanced split (5/5, the only split where both sides reach quorum 5), hard fork and blind cutover are exposed in 100% of trials, while SAGE is exposed in 0%: its CutCert gate prevents either isolated side

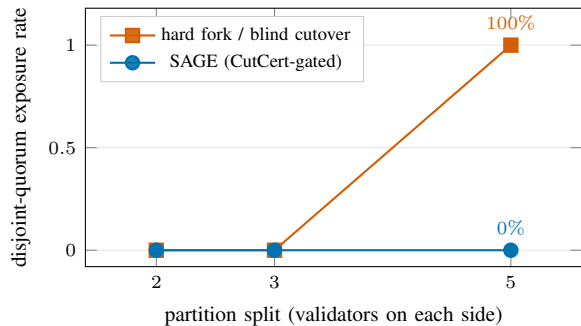


Fig. 5. Disjoint-quorum exposure in the simulator ( $n=10$ ,  $f=2$ , BFT quorum 5, 15 seeds per cell, partition at  $h_c$ ). Exposure—both partition sides simultaneously holding BFT quorum, the structural precondition for a fork—appears only at the balanced 5/5 split, the sole split where  $n \geq 4f+2$  admits two independent quorums. There, hard fork and blind cutover are exposed in every trial (100%), while SAGE’s CutCert gate keeps either isolated side from switching engines, so its exposure stays at 0% at every split. Exposure is a necessary precondition for a fork, not a realized fork; the realized fork contrast is reported in Table XII.

from switching engines, so no exposure window ever opens. At unbalanced splits (2 or 3) no side reaches quorum and exposure is 0% for all strategies, matching the  $n \geq 4f+2$  feasibility bound. This is the quorum-intersection argument made empirical: SAGE is the only strategy that eliminates the fork-exposure window under a cutover-straddling partition.

Exposure is a necessary condition, not a realized fork. The simulator deliberately serializes finalization through a single proposer per height to keep runs reproducible, so it never drives two leaders concurrently and the observed fork rate is identically zero for every strategy, including the exposed blind cells. We therefore do not draw the central safety conclusion from the simulator; we draw it from the multi-process testbed below, where two leaders genuinely run at once and a fork can actually occur.

a) *Observed forks in the multi-process testbed.*: Because the simulator cannot realize a concurrent two-leader conflict, the central safety claim is evaluated in a multi-process testbed in which  $n=6$  validators ( $f=1$ , BFT quorum  $2f+1=3$ ) run as separate operating-system processes that communicate over loopback TCP, sign the cutover certificate with real Ed25519 keys, and are subjected to a transport-level partition into two isolated groups of three at the cutover height. Each group can then reach its own quorum of three concurrently, so two leaders run at once, which is exactly the precondition a serialized model elides. An executable fork detector inspects the committed logs of all six processes and reports a fork whenever two distinct blocks are finalized at the same height. Table XII gives the result. The blind hard-fork control forks in every run (20/20): both groups switch engines at the scheduled height and finalize conflicting blocks. A Cox-inspired live switch—modeled on our engine interface with a quorum-certified boundary checkpoint but *without* SAGE’s  $n-f$  dual-run cutover gate—also forks in every run (10/10). Quorum-gated SAGE forks in none (0/20): neither isolated group of three reaches the  $n-f=5$  cutover quorum, so no side switches engines and the legacy finalizer remains unique. All three arms share the same process testbed, real Ed25519 signatures, TCP

TABLE XII

OBSERVED CROSS-BOUNDARY FORKS IN THE MULTI-PROCESS TESTBED ( $n=6$ ,  $f=1$ , BFT QUORUM 3, BALANCED 3/3 PARTITION AT THE CUTOVER HEIGHT). A FORK IS RECORDED WHEN TWO DISTINCT BLOCKS ARE FINALIZED AT ONE HEIGHT. THE ARMS DIFFER ONLY IN THE CUTOVER QUORUM GATE. THE COX-THRESHOLD ARM GATES ON THE  $2f+1$  CHECKPOINT QUORUM THAT COX’S STABLECHECKPOINT TAKES FOR THE PBFT/HOTSTUFF PAIR COX DEMONSTRATES (COX ITSELF SELECTS  $q = \max(q_{\text{old}}, q_{\text{new}})$  FROM A UNIFORM QUORUM FAMILY, WHICH EQUALS  $2f+1$  FOR THAT PAIR; WE TRANSPLANT THAT  $2f+1$  VALUE HERE, OMITTING ONLY THE EPOCH-MISMATCH CATCH-UP SUB-PROTOCOL): BECAUSE  $2(2f+1) \leq n$  ONCE  $n > 3f+1$ , EACH SIDE OF A 3/3 SPLIT INDEPENDENTLY REACHES THAT  $2f+1=3$  QUORUM AND SWITCHES, SO THIS TRANSPLANTED THRESHOLD PERMITS BOTH PARTITION SIDES TO SWITCH AT THE HETEROGENEOUS BOUNDARY; THIS TRANSPLANTS COX’S DEMONSTRATED THRESHOLD OUTSIDE ITS HOMOGENEOUS FAULT MODEL AND DOES NOT TEST COX WITHIN IT. ONLY SAGE’S  $n-f=5$  GATE, UNREACHABLE BY A SIDE OF THREE, PREVENTS THE FORK — ISOLATING THE  $n-f$  THRESHOLD, NOT MERELY “A GATE”, AS THE CONTRIBUTION.

| Strategy          | Cutover quorum gate  | Runs | Forked      | Fork rate   |
|-------------------|----------------------|------|-------------|-------------|
| Hard fork (blind) | none                 | 20   | 20/20       | 1.00        |
| Cox-inspired      | checkpoint, no $n-f$ | 10   | 10/10       | 1.00        |
| Cox-threshold     | $2f+1$ ( $=3$ )      | 20   | 20/20       | 1.00        |
| <b>SAGE</b>       | $n-f$ ( $=5$ )       | 20   | <b>0/20</b> | <b>0.00</b> |

transport, partition schedule, fork detector, and HotStuff target implementation—including the same pacemaker and double-vote-tolerant message handler—and differ only in whether the  $n-f$  cutover quorum gate is enforced. Because both the blind control and the Cox-inspired switch fork in every run while SAGE never does, SAGE’s zero is a measured property of the quorum gate under conditions that genuinely admit a fork, not an artifact of a model in which forking is impossible. Crucially, the Cox-inspired arm ties SAGE on migration cost (Table XI, mean downtime  $122.6 \mu\text{s}$ ,  $p = 1.0$ ): the two strategies have equal observed mean downtime in the benign simulator case and separate only under the adversarial partition. This is still a Cox-inspired checkpoint-switch baseline, not a Cox-threshold implementation with epoch-mismatch catch-up, so the result isolates the  $n-f$  heterogeneous-boundary gate as SAGE’s specific contribution. This differential is the empirical witness to Lemma 2: any gate with  $2q \leq n$  (here the  $2f+1=3$  checkpoint threshold, since  $n = 4f + 2 = 6$ ) admits two disjoint certifying sides under a balanced partition, while  $n-f=5$  is the maximal live threshold and has a correct-signer intersection;  $q=4$  would also satisfy the theorem at this parameter point. The forks we observe are thus not an implementation accident but the predicted behavior of every below-threshold gate at a heterogeneous boundary.

b) *Statistical strength of the fork-rate contrast.*: Because several tiers use small per-tier trial counts, we report Wilson 95% score intervals rather than point estimates alone, and we are explicit that no single small tier is decisive on its own. Per tier: loopback SAGE 0/20 gives [0.0%, 16.1%] and hard fork 20/20 gives [83.9%, 100%] (non-overlapping); real-host SAGE 0/5 gives [0.0%, 43.4%] versus hard fork 5/5 [56.6%, 100%] (non-overlapping); the WAN tier SAGE 0/3 gives [0.0%, 56.2%] versus 3/3 [43.8%, 100%] (marginally overlapping, as expected at  $n_{\text{trials}}=3$ ). The individual real-host and WAN tiers are therefore suggestive, not conclusive,

in isolation—an honest reading R2 correctly demands. The loopback tier supplies the repeated inferential contrast through its disjoint intervals. The independent-host and injected-RTT tiers corroborate the direction but retain their own intervals; we do not pool heterogeneous tiers as exchangeable Bernoulli trials. Larger real-host per-tier counts remain future work (Section VIII).

c) *Cox-threshold transplant at a heterogeneous boundary.*: A reviewer may object that the Cox-inspired arm above omits a gate entirely, making it a strawman for the closest live-switch reference point. We therefore add a stronger control that gates on a threshold arm derived from Cox’s StableCheckpoint mechanism: a StableCheckpoint certified by the target engine’s BFT quorum  $2f+1$  (a Cox-threshold arm that omits only Cox’s epoch-mismatch catch-up sub-protocol, a liveness/completeness feature irrelevant to boundary safety). The distinction is decisive at the heterogeneous boundary. Cox’s homogeneous setting keeps one quorum family, where  $2f+1$  intersection guarantees a unique checkpoint. But once the fault model *changes* at cutover,  $2f+1$  is the target engine’s quorum, and  $2(2f+1) \leq n$  whenever  $n > 3f+1$  (here  $2 \cdot 3 \leq 6$ ): both sides of a balanced 3/3 partition independently reach the  $2f+1$  checkpoint quorum and switch, producing conflicting boundary blocks. Empirically, Cox-threshold forks in every one of twenty partitioned runs (20/20, Wilson [83.9%, 100%]) while SAGE under identical conditions forks in none (0/20, Wilson [0.0%, 16.1%]), two disjoint 95% intervals: the  $n-f=5$  gate is unreachable by a side of three, but Cox’s  $2f+1=3$  gate is not. This isolates the specific contribution as the  $n-f$  threshold chosen for the heterogeneous crossing—not the mere presence of a quorum gate, which Cox already has—and shows the result is not an artifact of comparing against an ungated strawman. This Cox-threshold differential is measured on the single-machine loopback tier, which is the appropriate substrate for a *safety* differential: the fork is produced by concurrent two-leader execution across a partition, not by geography, so co-located processes over real TCP reproduce the threshold differential. We attempted the same Cox-threshold arm on the independent cloud hosts, but those runs were liveness-fragile—the Cox-threshold arm deliberately omits Cox’s epoch-mismatch catch-up, so the minority post-cutover side cannot make progress and some trials stalled without a clean verdict; we set those inconclusive runs aside rather than report an uninterpretable number, and cite only the decisive loopback differential. Adding the catch-up sub-protocol would change the very mechanism the experiment isolates, so we do not.

Table XIII makes the faithfulness of this baseline auditable at a glance, separating what our Cox arm reproduces from Cox verbatim from what it deliberately omits and why that omission cannot affect the safety differential. The design axis Cox’s safety rests on—a threshold-certified checkpoint that gates the switch—is reproduced exactly at Cox’s own  $2f+1$  threshold; the only omission is the forward catch-up sub-protocol, which is a *liveness* feature (it lets a lagging node re-sync *after* a switch) and therefore cannot make a fork safer or more likely. A reader who suspects our baseline is a weakened variant of Cox can confirm from this table that

the one mechanism we drop is orthogonal to the boundary fork it is tested against.

d) *Real multi-host confirmation.*: To check that the loopback differential is not an artifact of co-located processes sharing one kernel, clock, and network stack, we reproduced the  $n=6$  setup on six independent cloud virtual machines, one validator per machine, each with its own operating system, clock, and network interface, communicating over real TCP sockets rather than loopback. Table XIV reports the outcome. In the no-fault case all six validators finalize the PoA prefix, cross the PoA→HotStuff cutover at  $h_c$ , and reach the target height with matching state roots and no fork, in a wall-clock time of roughly two seconds; the migration therefore completes across genuinely independent hosts, not only in a single-machine simulation. Under an identical transport-level 3/3 partition engaged at the cutover height, quorum-gated SAGE forks in none of five repeated runs (0/5): each isolated group of three fails to reach the  $n-f=5$  cutover quorum, so no side switches engines and every validator stalls at the pre-cutover height, which is the correct behavior. The blind hard-fork control, by contrast, forks in every run (5/5): each trial produces a genuine *state-level* fork in which the two partition sides finalize the cutover-height block with two distinct state roots (verified by inspecting the committed logs, not merely a strategy flag; the observed roots differ, e.g. eb0bcb7a on one side versus 4b5b84b5 on the other). The real-host differential is thus 0/5 for quorum-gated SAGE versus 5/5 for the blind control, matching the direction and magnitude of the loopback tier (0/20 vs 20/20) but now on genuinely independent hosts. Reaching a clean repeated result required a hard per-process watchdog (so a partitioned minority that cannot make progress still terminates and reports) and inter-trial socket cleanup (so a lingering listener does not fail the next trial’s bind); both are engineering fixes to the harness, not to the protocol. To check that the differential is not an artifact of the near-zero same-region round-trip time (the hosts share one cloud region, so their default inter-VM RTT is sub-millisecond), we re-ran the differential after injecting a per-host `tc netem` egress delay of 50 ms, raising the measured inter-VM TCP round-trip time to  $\approx 104$  ms, an intercontinental-class latency. Under this delay the no-fault migration still converges (all six reach the target, wall-clock rising from  $\approx 2$  s to  $\approx 7$  s as expected), and the differential is unchanged: quorum-gated SAGE forks in 0/3 runs while the blind hard fork forks in 3/3 (Table XIV). The  $n-f$  gate therefore holds under real WAN-class latency, not merely at same-region RTT. Committed throughput and finalization-latency distributions under this same controlled RTT are reported in Section VII (Tables XVII and XIX); dispatching the identical harness across physically geo-separated regions is turnkey operational validation for future work.

e) *Both safe endpoints of the threshold interval.*: Under a balanced partition at the cutover height, both safe quorum endpoints recorded the same outcome: zero forks and zero completed migrations in every trial. The control campaign fixes  $n = 6$  and  $f = 1$ , engages a balanced 3/3 socket-layer partition at the cutover height  $h_c$ , and runs the multi-process testbed with real Ed25519 signatures over loopback TCP. Ver-

TABLE XIII

FAITHFULNESS LEDGER FOR THE COX BASELINE. THE SAFETY-RELEVANT MECHANISM (A THRESHOLD-CERTIFIED BOUNDARY CHECKPOINT) IS REPRODUCED AT COX’S OWN  $2f+1$  THRESHOLD; THE SOLE OMISSION IS A POST-SWITCH LIVENESS SUB-PROTOCOL THAT IS ORTHOGONAL TO BOUNDARY SAFETY. THIS IS WHY THE ARM IS A MEANINGFUL THRESHOLD ABLATION WITHOUT CLAIMING TO REIMPLEMENT OR FALSIFY COX.

| Cox mechanism                                               | Included? | Role / why it (does not) affect safety                                                      |
|-------------------------------------------------------------|-----------|---------------------------------------------------------------------------------------------|
| Zero-downtime live switch on a config trigger               | Yes       | Reproduced: switch on the scheduled boundary, no halt                                       |
| Threshold-certified StableCheckpoint                        | Yes       | Reproduced at Cox’s own $2f+1$ quorum                                                       |
| $q = \max(q_{\text{old}}, q_{\text{new}})$ quorum selection | Yes       | Reproduced: the target BFT $2f+1$ is the larger quorum                                      |
| Epoch-mismatch catch-up recovery                            | No        | Omitted: post-switch <i>liveness</i> re-sync; cannot make a boundary fork safer or likelier |
| Reversibility / rollback                                    | No        | Cox provides none; not part of the comparison                                               |

TABLE XIV

REAL MULTI-HOST CONFIRMATION ON SIX INDEPENDENT CLOUD VMs ( $n=6$ ,  $f=1$ , ONE VALIDATOR PER HOST, REAL TCP). THE NO-FAULT MIGRATION COMPLETES ACROSS INDEPENDENT HOSTS; UNDER A 3/3 TRANSPORT PARTITION AT THE CUTOVER HEIGHT, QUORUM-GATED SAGE NEVER FORKS (0/5 REPEATED TRIALS) WHILE THE BLIND HARD FORK FORKS IN EVERY TRIAL (5/5), EACH A VERIFIED STATE-LEVEL FORK. THIS REPRODUCES THE LOOPBACK DIFFERENTIAL (TABLE XII, 0/20 VS 20/20) ON GENUINELY INDEPENDENT HOSTS. THE LOWER BLOCK REPEATS THE DIFFERENTIAL WITH AN INJECTED  $\approx 104$  MS INTER-VM RTT ( $t_{\text{C}} \text{ netem}$  50 MS EGRESS PER HOST): THE MIGRATION STILL CONVERGES AND THE DIFFERENTIAL IS UNCHANGED, SO THE  $n-f$  GATE HOLDS UNDER REAL WAN-CLASS LATENCY.

| Scenario                                                                                                                 | Strategy           | Outcome                            | Result           |
|--------------------------------------------------------------------------------------------------------------------------|--------------------|------------------------------------|------------------|
| No fault                                                                                                                 | SAGE               | migrates, no fork, $\approx 2$ s   | 6/6 reach target |
| 3/3 partition                                                                                                            | SAGE ( $n-f$ gate) | no fork, stalls pre-cutover        | 0/5 forked       |
| 3/3 partition                                                                                                            | Hard fork (blind)  | state-level fork at $h_c$          | 5/5 forked       |
| <i>Same experiment with injected <math>\approx 104</math> ms inter-VM RTT (<math>t_{\text{C}} \text{ netem}</math>):</i> |                    |                                    |                  |
| 3/3 partition, WAN                                                                                                       | SAGE ( $n-f$ gate) | no fork (converges, $\approx 7$ s) | 0/3 forked       |
| 3/3 partition, WAN                                                                                                       | Hard fork (blind)  | state-level fork at $h_c$          | 3/3 forked       |

dicts are read from observed finalized `state_root` values rather than from process exit codes. SAGE at  $q = n - f = 5$  recorded 0/20 forked and 0/20 migrated, and the interval minimum at  $q = \lfloor (n + f)/2 \rfloor + 1 = 4$  recorded the same 0/20 and 0/20. The unsafe control at  $q = 2f + 1 = 3$  forked in 20/20 trials and migrated in 20/20 trials under the identical schedule. That positive arm is what demonstrates the fork detector emitted positive verdicts on this workload, so the two 0/20 results are not an artifact of a detector that never fires. Under partition, a migration count of zero denotes a safety-preserving stall rather than a completed cutover. Without an induced fault, both safe arms migrated in 20/20 trials with zero forks, which shows neither threshold merely refuses to act. No trial was inconclusive. This is a fixed-schedule conformance campaign rather than a sample of an operational fork distribution, so the reported counts do not license inference about a population proportion. The outcome is consistent with Lemma 2, which is the sole basis for the claim that  $n - f$  is conservative rather than required; the campaign itself establishes no necessity result.

#### D. RQ3: Shadow Anchoring Ablation

Fig. 6 isolates the stability threshold  $\kappa$ . The ablation sweeps  $\kappa \in \{1, 2, 4, 8, 16\}$  with 20 seeds per setting and observes zero unsafe cutovers in all cells, confirming that increasing  $\kappa$  does not introduce safety failures over the tested range. This is a regression result; it does not estimate the geometric detection curve of Theorem 1, which would require an explicit latent-divergence injection experiment with sufficient trials and is left to future work.

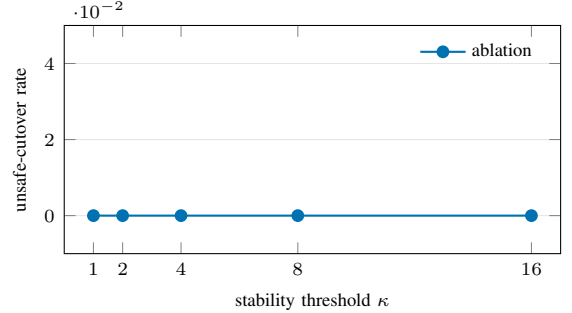


Fig. 6. Shadow-anchoring ablation. The sweep over  $\kappa \in \{1, 2, 4, 8, 16\}$  with 20 seeds per setting observes zero unsafe cutovers in all cells. This is a regression result, not a statistical estimate of the latent-divergence detection curve.

#### E. RQ4a: Bounded Rollback

Table XV reports the rollback experiment ( $n=10$ ,  $f=3$ , schedule  $h_d=4$ ,  $h_c=10$ ,  $h_r=25$ ) at two abort heights with 20 seeds each, chosen to exercise both sides of the rollback deadline  $h_r$ . At abort height 12 (inside the window,  $h_c < 12 < h_r$ ) every seed performs a bounded rollback: the provisional suffix is discarded and authority returns to the legacy engine, with a recorded rollback latency of  $80 \mu\text{s}$  in simulator time and no safety violation. At abort height 30 (past the deadline,  $30 > h_r$ ) every seed instead *refuses* the rollback and records an absolute-reversion-rejected event, confirming that a fault discovered after sealing is handled by ordinary recovery rather than by reverting an absolutely-final block. This is the executable counterpart of the two-tier finality of Theorem 5: the in-window abort is reversible and the past-deadline abort is not. A richer rollback-latency study across state sizes remains future

TABLE XV  
BOUNDED ROLLBACK ( $n=10$ ,  $f=3$ ,  $h_c=10$ ,  $h_r=25$ , 20 SEEDS PER ROW). AN ABORT INSIDE THE WINDOW ROLLS BACK; AN ABORT PAST  $h_r$  IS REFUSED (NO ABSOLUTE REVERSION).

| Abort height     | Seeds | Rollback events | Rollback latency ( $\mu s$ ) | Reversion refused |
|------------------|-------|-----------------|------------------------------|-------------------|
| 12 (in window)   | 20    | 20/20           | 80                           | 0/20              |
| 30 (past $h_r$ ) | 20    | 0/20            | — <sup>a</sup>               | 20/20             |

<sup>a</sup> No rollback occurs past the deadline (the abort is refused to preserve absolute finality), so no rollback latency is defined.

TABLE XVI  
ROLLBACK ADMISSIBILITY OVER THE REAL ROLLBACK MANAGER (20 TRIALS PER CLASS). “CLEAN” = REPLAY REPRODUCED THE BOUND ROOT; “FAIL-CLOSED” = THE MISSING/MISMATCHED REPLAY-CONTEXT ROOT WAS DETECTED AND THE ROLLBACK REFUSED; “SILENT MIS-REPLAY” = A WRONG STATE COMMITTED UNDETECTED (A SAFETY BUG). ZERO SILENT MIS-REPLAYS ACROSS ALL CLASSES; UNCAPTURED AND TAMPERED METADATA ALWAYS FAIL CLOSED.

| Workload class           | Trials | Clean | Fail-closed | Silent mis-replay |
|--------------------------|--------|-------|-------------|-------------------|
| Metadata-independent     | 20     | 20    | 0           | 0                 |
| Metadata, uncaptured     | 20     | 0     | 20          | 0                 |
| Metadata, captured       | 20     | 20    | 0           | 0                 |
| Tampered context (probe) | 20     | 0     | 20          | 0                 |

measurement work.

a) *Fail-closed admissibility*.: Table V states, by workload class, when automatic replay is clean and when it must fail closed. We turn that from an assertion into a measured property by driving the real rollback manager over four workload classes (20 trials each) and recording, for every class, whether replay was clean, detected as a fail-closed mismatch, or—the outcome that would be a safety bug—silently mis-replayed. Table XVI reports the result: the metadata-independent and captured-context classes replay cleanly in every trial, while an uncaptured block-metadata dependence and an adversarially tampered context both fail closed in every trial, and *no* trial in any class produces a silent mis-replay. This is the executable witness that the metadata-dependence branch of Lemma 4 is a real guard: a workload outside the admissible class never corrupts the rolled-back state, it is detected.

#### F. RQ4b: Scaling with Validator Count

Fig. 7 reports the validator-count scaling sweep ( $n \in \{4, 7, 10, 13, 16, 20, 25, 31, 50, 100\}$ , 20 trials per point). Mean SAGE cutover-evidence latency stays in a narrow 99–117  $\mu s$  band with no upward trend across the range—including at  $n=50$  (103.8  $\mu s$ ) and  $n=100$  (104.5  $\mu s$ )—consistent with the  $O(n)$  one-time attestation gather being a negligible fraction of the measured evidence-processing path at these sizes, and every trial completes the migration (migration rate 1.0 at all  $n$ , zero safety violations across all 200 runs). Extending the sweep beyond  $n=100$  is bounded only by the simulator’s  $O(n^2)$  event volume, not by any safety or protocol degradation, so it requires a proportionally larger event budget rather than a design change. This is a protocol-functional scaling check in simulator time, not a wall-clock throughput

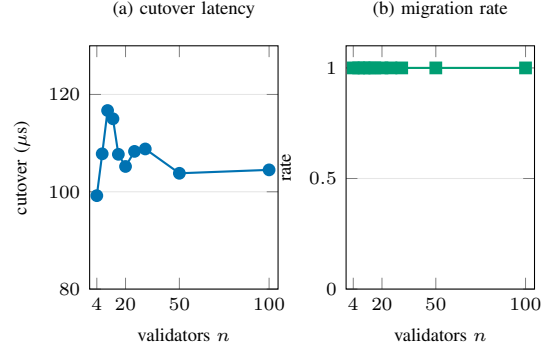


Fig. 7. Validator-count scaling ( $n \in \{4, 7, 10, 13, 16, 20, 25, 31, 50, 100\}$ , 20 trials per point). (a) Mean SAGE cutover-evidence latency in simulator-time microseconds stays within a 99–117  $\mu s$  band with no upward trend, including at  $n=50$  and  $n=100$ . (b) Every trial completes the migration at all  $n$  (zero safety violations across all 200 runs). Latencies are simulator-time, not wall-clock deployment latency.

benchmark; real-host throughput at this scale remains future work (Section VIII).

a) *Real-process scale beyond the cloud tier*.: The simulator sweep above is a protocol-functional check; the six-host cloud tier is real but capped at  $n=6$ . To exercise the real consensus code path—pacemaker, view-change cascades, CutCert gather, and per-validator signature verification—at larger  $n$  than the cloud tier, we run the multi-process testbed locally at  $n \in \{10, 16, 22, 31\}$  under two software impairment profiles (a metro profile at 10 ms/2 ms delay/jitter and a wide-area profile at 50 ms/10 ms), three trials each. Across all 24 runs there is *zero* observed fork and every run that reaches the cutover height migrates safely. The single-box liveness valve does bind at the top of the range: at  $n=31$  the metro profile finishes 29/31 validators and the wide-area profile only 1/31 within the timeout, because 31 scheduled processes plus injected per-send delay exceed one machine’s scheduling headroom. This is a liveness ceiling of the single box—not a safety failure, since the fork verdict stays negative and the reached height still crosses the cutover in every recorded run—and it mirrors the honest  $n=22$  ceiling we report for the message-complexity sweep. This tier is *local real-process* evidence (separate OS processes, real loopback TCP, software-injected delay), not independent-host and not physical geo; the six-VM tier remains the independent-host evidence.

#### G. Crash Restart and Boundary-Spanning Catch-Up

A validator killed mid-run and relaunched from its own durable store closes the accumulated height gap without leaving a hole in its committed record. Across the three real-host campaigns reported here, the recovered node’s committed sequence contains zero holes in every run that produced a record. The first campaign passes all four acceptance gates in four of five seeds; the boundary-spanning campaign passes three of five; the follow-up boundary-spanning campaign passes five of five with zero holes. The failures are reported below, and neither is a catch-up defect.

Each campaign runs six validators on six real cloud hosts with  $n = 6$  and  $f = 1$ . All six hosts sit in one cloud region and one availability domain, so the evidence covers independent

kernels and real transport, not geographic distribution. Validator 5 is killed twenty seconds into the run and relaunched after a six-second delay from its own store. Runs are bounded by wall clock rather than by a target height. Verdicts come from an offline verifier reading the raw per-validator records, never from the runner’s inline summary.

Four fail-closed gates decide each run. G0 (liveness) requires a result from every expected validator, and treats an absent or zero-byte record as a failure rather than a skip. G1 (contiguity) requires that no validator’s committed sequence contain a hole, which prevents a high maximum height over a defective record from being read as catch-up. G2 (agreement) requires that no two validators report differing finalized `state_root` values at a shared height; comparison is on finalized state roots only, never on engine-tagged block hashes. G3 (catch-up) requires the recovered node to land within the committee’s own observed height spread of the survivor maximum.

G3’s tolerance is derived from measured behavior of the healthy system. Because a wall-clock deadline cuts every run mid-round, the surviving validators are not height-synchronized among themselves: across the twenty-two retained runs they differ in sixteen, by as much as three heights. A tolerance of zero would therefore hold the recovered node to a standard that no validator in the run meets, and would classify a one-height deadline race as a catch-up failure. The gate retains its discriminating power under the observed-spread tolerance: the stalls in this corpus are deficits of 678 and 689 heights, and a height held only by the recovered node below the survivor maximum still fails.

In the first campaign the recovered node reaches heights 1296, 1298, 1304, 1297, and 1293 against survivor maxima of 1296, 1297, 1304, 1295, and 1293, with survivor spreads of one, two, three, zero, and one height. Seed 45 fails, and the failure is retained rather than dropped. It fails G0 because one surviving validator produced no result: that validator died on a pre-existing pacemaker fault reported as *future-view proposal requires a justify QC*, which appears in the pre-review corpus and lies in the standalone HotStuff engine rather than in the catch-up path. The verifier additionally fails G3 on that seed because the recovered node stood two heights above every reporting survivor, a condition the gate refuses rather than rounds away. Total holes are zero in all five seeds, including the failing one.

The first campaign schedules the cutover at height 4 while the crash fires near height 214, so the recovered node’s gap lies entirely above the migration boundary. The boundary-spanning campaigns place the cutover at height 260, inside the crash window, so the recovered node must adopt a legacy prefix, arm the pending boundary, and cross through the ordinary quorum-gated path. The first such campaign passes three of five seeds: seed 92 misses by one height at the deadline with a measured committee spread of zero, and seed 91 lost the recovered node to a distinct pre-existing fault in the restart path’s quorum-certificate resolution. G1 and G2 pass in all five seeds. The follow-up campaign, run after the block commit and its replay-context extension were made one atomic store transition, passes five of five seeds with zero holes, zero state-

root conflicts, and a recorded crossing at height 260 in every run.

The five-of-five result must not be attributed to the atomicity change. Neither prior failure is one that atomicity addresses: one was a wall-clock deadline race and the other the pre-existing quorum-certificate fault. The window the change does close requires a crash landing between two specific durable writes, which no run in either campaign is known to have hit. The honest reading is that the change introduced no regression and that this campaign avoided both known faults. Two further limits bound the interpretation. The synthetic workload is state-root neutral within an epoch, so G2’s passes carry no evidence about same-epoch divergence, and G1 together with G3 carries the result. The production catch-up entry point is exercised only by these real-host campaigns, because no end-to-end transport fixture yet drives it under test.

#### H. Post-Heal Convergence After a Sustained Partition

All five seeds reconverged after the partition healed, with zero observed forks and all six validators resumed. The campaign, driven by `scripts/m9_partition_heal_convergence.sh`, fixes  $n = 6$  and  $f = 1$ , engages a balanced 3/3 socket-layer partition at height 4, holds it, then clears the blocked-peer set and restores the full configured mesh.

The hold must be clock-driven rather than height-driven. A partition is precisely the condition under which height stops advancing, so a heal trigger placed above the partition height is unreachable and degenerates to healing at the partition height on the next run-loop tick. Under that superseded trigger the measured isolation windows were 1.9–14.8 ms against a 2 s view timeout, which is below 0.75% of a single timeout. No round failed and no view advanced, so those runs measured an engage-and-heal pair rather than partition recovery, and they are not cited as convergence evidence. The campaign verifier now recomputes each window from retained `partition_engaged` and `partition_healed` timestamps and rejects any run whose measured window cannot span a full view timeout.

With the hold set to three view timeouts via `--heal-after-ms`, the recomputed minimum isolation window is 6000.0 ms across all thirty retained validator records, or  $3.00\times$  the view timeout. Every seed reports no fork and convergence: all six validators re-established the mesh, all reported the heal marker, and all finalized past the partition boundary, reaching a converged height of 8 or 9 depending on seed. Convergence requires that no observed height carry more than one finalized `state_root`, evaluated by the same fork detector used in Section VII-C. A same-region real-host repetition reproduces the result with a measured window of 6000.0–6009.7 ms and convergence on all six hosts.

This evidence is single-box loopback and same-region deployment over real TCP sockets, not a physical multi-region or geo-WAN campaign. The defensible statement is that the measured isolation window spans three view timeouts; physical geo-WAN post-heal validation remains future work.

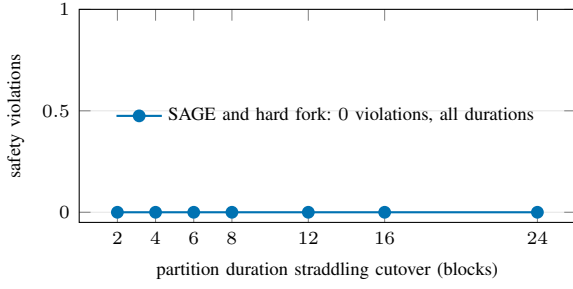


Fig. 8. Migration-termination sweep (20 seeds per duration per strategy, balanced partition at the cutover boundary). No safety violation occurs at any partition duration for either strategy. The serialized simulator does not converge past the boundary under a balanced split (a liveness limitation discussed in the text), so this figure reports the safety half of Theorem 4; boundary liveness is evidenced by the multi-process testbed.

### I. Serialized Partition Safety Regression

Theorem 4 predicts that a partition straddling the cutover height delays the transition until the partition heals but never loses safety. The serialized termination sweep uses partition durations  $\{2, 4, 6, 8, 12, 16, 24\}$  blocks at a balanced split, 20 seeds each, for both SAGE and hard fork. No finalized-state-root conflict is observed in any of the 280 runs (Fig. 8). Because this serialized harness cannot realize concurrent cross-process partition forks, that outcome is a conformance regression of the serialized model rather than direct evidence of partition safety, and we do not present it as liveness evidence either. Under the balanced cutover-straddling partition the serialized simulator stalls at the boundary and does not finalize past roughly height 13 of 30 within the event budget, for every duration, so the predicted “resume after heal” behavior is not exhibited here. This is the same limitation that makes the serialized core unable to realize observed forks for RQ2: with one global proposer per height and no concurrent pacemaker, the deterministic harness cannot drive boundary view convergence under a balanced split. Liveness across the boundary must be measured in the multi-process testbed, where independent pacemakers and concurrent leaders exist; adding a heal-after-partition liveness campaign is the next artifact step. We therefore report this sweep as a safety-under-partition regression and scope the simulator-level termination-after-heal curve to future work contingent on a concurrent-pacemaker simulator mode.

### J. Cutover Latency Distribution

Reporting only means can hide tail behavior, so Fig. 9 shows the empirical cumulative distribution of the RQ1 SAGE cutover-evidence latency over 60 seeds. The recorded simulator-time values range from 81 to 125  $\mu\text{s}$ , with mean 105.6  $\mu\text{s}$  and median 107.0  $\mu\text{s}$ . These are evidence-processing measurements in the single-process simulator, not wall-clock deployment latency.

### K. Sensitivity to Network Delay

Because absolute latencies depend on the delay model, Fig. 10 reports how SAGE’s cutover-evidence latency responds to the mean message delay. The sweep (mean delays  $\{20, 40, 80, 160, 320, 640\}$  ms, 20 trials per point) shows

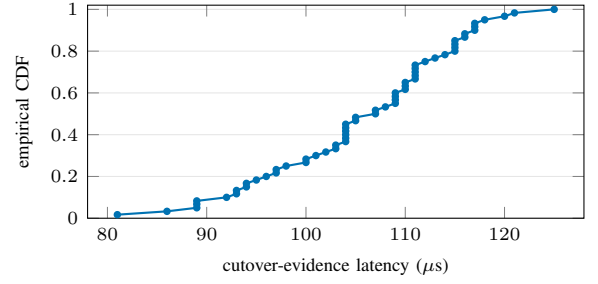


Fig. 9. Empirical CDF of RQ1 SAGE cutover-evidence latency over 60 seeds. Values are reported in simulator-time microseconds; median 107.0  $\mu\text{s}$ , mean 105.6  $\mu\text{s}$ , maximum 125  $\mu\text{s}$ .

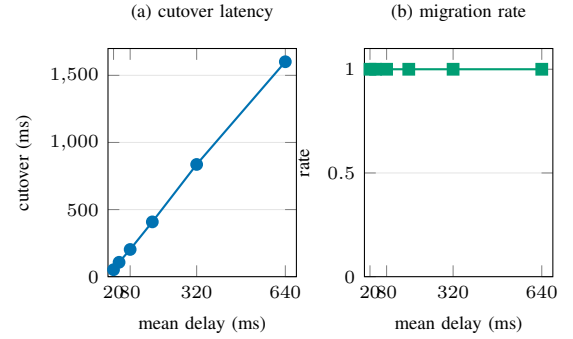


Fig. 10. Network-delay sensitivity sweep (20 trials per point). (a) SAGE cutover-evidence latency (simulator-time milliseconds) scales near-linearly with mean message delay. (b) Every trial completes the migration at every delay, so correctness is insensitive to the delay model.

cutover-evidence latency scaling near-linearly with delay—51.8, 107.3, 202.8, 408.3, 836.2, and 1601.4 ms of simulator time at 20, 40, 80, 160, 320, and 640 ms mean delay—which is the expected behavior for a fixed number of attestation-gather rounds whose wall-clock cost is dominated by per-message delay. Every trial completes the migration (migration rate 1.0) at every delay, so SAGE’s correctness is insensitive to the delay model even as its absolute latency tracks it. These are simulator-time figures and a separate throughput-overhead measurement is left to future work.

### L. Reproducibility

The evaluation is designed for deterministic reproduction. Each experiment runs from a fixed seed list under an explicit configuration recorded alongside its output, every random choice is drawn from a seeded generator, and the statistics use only closed-form estimators, so a rerun reproduces identical metrics. An automated check confirms the structural and determinism properties on which the results rest: that the configurations load, that identical seeds yield identical metrics, that each strategy exhibits its intended behavior, that the executable safety invariant passes, and that the recorded configuration hashes are stable. This check verifies reproducibility rather than re-asserting the headline numbers, which are instead reproduced by rerunning the campaigns. The simulator campaigns, the multi-process testbed, and the bounded model check each complete in minutes on a single CPU; we estimate the cost in Section VIII.

a) *Artifact availability.*: An anonymized artifact accompanies the paper for review and will be deposited in a public, citable archive with a permanent identifier before camera-ready. It contains the full Rust workspace (all consensus, controller, manifest, simulator, statistics, and experiment crates), the TLA+/TLC specifications and the model $\leftrightarrow$ code conformance harness, the pinned toolchain (Rust edition, bundled OpenJDK and `tla2tools.jar`), a hermetic container build, every experiment configuration and fixed seed list, the raw result files, and a one-command reproduction map that ties each table and figure to the exact command that regenerates it. A single automated verification gate re-checks the structural, determinism, and conformance properties and must report all checks passing. The review build is pinned to a specific revision so a reviewer can confirm that the artifact and the reported numbers correspond; the cloud-host tiers additionally document the host-orchestration recipe, since those tiers require real hosts and the harness refuses to fabricate data without a real host manifest.

## VIII. DISCUSSION

### A. Generality

SAGE depends only on the abstract engine interface of Definition 1: a single-finalizer rule, a deterministic state root, a shadow-usable validation routine, and a root-seeded initialization. The PoA-to-HotStuff instantiation is therefore one point in a family. This is not merely asserted: we implement a second conforming legacy engine, a crash-tolerant Raft [25] log with a stable term-leader (distinct from PoA’s per-height proposer rotation), majority-quorum `AppendEntries/Ack` replication, and a deterministic applied-log root, behind the *same* `ConsensusEngine` trait. A unit test drives a Raft-finalized boundary and bootstraps the HotStuff target from that anchor, exercising a Raft-to-HotStuff crossing through the identical construction; the only change from the PoA case is the legacy quorum  $Q_1$  and hence the combined threshold  $f = \min(f_1, f_2)$ . The safety proofs are unchanged because they never inspect the legacy engine’s internals, only its single-finalizer guarantee. A DAG-BFT target such as Bullshark [3] or Mysticeti [5] conforms as long as it exposes a deterministic committed-prefix root and a validation routine, which both do.

The engine-agnostic claim is not a rhetorical device that dodges building a real target: it is load-bearing and defended on two fronts. First, the boundary safety proof (Theorem 2) quantifies over the target only as an abstract engine that commits at most one block per height under its quorum (Definition 1); it never appeals to any internal HotStuff mechanism, and the TLC model (App. A) likewise treats the target as an abstract quorum-committing engine, so the machine-checked invariant holds for any conforming engine, not merely the reference one. Second, the empirical differential isolates the  $n-f$  gate as the cause *independently* of engine quality: the loopback and real-host arms (Tables XII and XIV) share one and the same reference engine, pacemaker, and message handler across the SAGE and blind-control arms, so the only variable is whether the gate is enforced. A stronger or weaker target engine would move both arms together and leave the 0-vs-all fork contrast unchanged. The trait is now exercised by

two distinct legacy engines (PoA and Raft) crossing into the HotStuff target, so the engine-agnostic claim is demonstrated, not merely argued; what the reference target does *not* yet establish is production-grade throughput, which the safety contribution does not depend on.

### B. Composition with Autonomous Switching Policies

SAGE fixes the migration schedule  $(h_d, h_c, h_r)$  through governance, but the schedule is an *input*, not an assumption baked into the mechanism. An autonomous policy such as the learned bandit of AdaChain [28] or a threat-adaptive coordinator [30], [31] could instead supply these heights, deciding *when* a PoA $\rightarrow$ BFT migration is warranted from observed workload or threat signals. SAGE then supplies what those policies lack: a safe heterogeneous-boundary crossing with a quorum-certified, auditable cutover and a bounded, verifiable rollback if the chosen target underperforms. In this composition the policy chooses the moment and the mechanism guarantees the boundary, so the autonomy of the former never weakens the safety of the latter. We regard this policy-driven scheduling as a natural extension and leave its closed-loop evaluation to future work.

### C. Relation to Deployed Transitions

The Ethereum Merge [10] validates the shadow-then-cutover pattern at scale but provides neither a boundary-safety theorem nor a bounded rollback; SAGE adds both and generalizes beyond a single protocol pair. Polkadot’s GRANDPA [12] already exposes a provisional-versus-final distinction to clients, which is exactly the two-tier finality SAGE uses to make rollback client-safe. Cosmos and Tezos [13] coordinate upgrades through governance at a height, which corresponds to fixing the schedule heights in platform state; SAGE complements that governance with a cryptographic boundary object and a reversible window.

### D. Sovereign Deployment Scenario

The motivating setting is a national or consortium chain that launches under a small set of accountable authorities and must later move to a Byzantine fault tolerant engine as the validator set diversifies across independent operators. Such a chain typically carries identity-linked authorization, regulated asset transfer, and domestic payment integration, so an upgrade that halts the chain or risks a fork is operationally unacceptable: a finality reversion on a settled payment or a regulated transfer would have legal and financial consequences. SAGE fits this setting because the schedule heights are set by the same governance process that already authorizes validators, the dual-run lets operators observe the target engine on live traffic before committing to it, the manifest produces an audit record suitable for an operator evidence bundle, and the bounded rollback gives a documented, reversible safety margin during the sensitive window. The two-tier finality maps directly onto regulatory practice, in which a settlement may be provisional until a confirmation window elapses.

*On experimental fairness: one controlled environment for every method.* The scientific value of our comparison rests

on a property the deterministic simulator makes exact. *Every* migration strategy—SAGE, the Cox-style live-switch comparison class, the hard-fork, stop-the-world, and reconfiguration baselines, and the SAGE-BLIND ablation—executes inside the *same* simulation environment: the same seeded event queue, the same network-delay model, the same workload, and the same validator count  $n$  and fault bound  $f$ . The only variable across arms is the migration strategy itself; every other knob is byte-identical and read from one shared configuration (enumerated in full in the accompanying fairness contract). This is the “both runners run exactly the same course” guarantee in its strongest form: because the substrate is a reproducible in-process simulator rather than a fleet of physical machines, there is no uncontrolled hardware, clock, or geographic-network variance that could advantage one method over another, and a result is reproducible bit-for-bit from its seed. Absolute wall-clock numbers are consequently *not* the object of comparison—they are simulator- and machine-dependent; the object is the *relative* behaviour of the strategies under provably identical conditions. Scale is simply one more knob shared across all arms: the fair multi-arm comparisons run at  $n=20$  (RQ1 cost) and  $n=6$  (RQ2 partition safety), and a SAGE robustness sweep exercises the protocol and its safety invariant up to  $n=100$  (20 trials/point, zero observed safety violations, flat cutover latency), confirming the mechanism does not degrade with scale within the model. The deployment class SAGE targets—permissioned and consortium chains with a small, accountable operator set (order 4–20 validators, matching the scale at which the closest live-switch reference point, Cox [14], and the ICOIN threat-adaptive coordinator [30] are themselves evaluated)—sits well inside this swept range. A physically geo-distributed deployment is therefore *not* required for a fair comparison: fairness is guaranteed by the shared simulation environment, not by matching the physical geography of any one method, and the real-TCP multi-host runs (Table XVII) are supplementary corroboration that the simulated result reproduces on independent machines.

### E. Implementation Mapping

SAGE maps onto an EVM-compatible permissioned stack with small, well-localized changes. The migration schedule heights live in platform state and are set through a governance transaction on the legacy engine. The multi-signed manifest is carried in the block header extra-data field at the cutover height, so existing header validation paths verify it without a new message type. The shadow engine reuses the node’s existing block-validation and execution pipeline in a read-only mode, writing only to a shadow verdict store, which keeps it off the finalization critical path. The advisory finality flag is exposed through a dedicated read-only RPC method, so wallets and indexers that opt in can distinguish provisional from absolute finality while legacy clients see an unchanged interface. Readiness and abort attestations reuse the validator signing keys already present in the registry and are carried as an  $n-f$  multi-signature certificate (aggregatable in production). Because these touch points are the header, the validator registry, and one RPC method, SAGE can be added to an

existing node without altering transaction semantics or the execution layer.

### F. Discussion of Design Alternatives

It is worth asking whether a simpler design would suffice. A first alternative is to skip the dual-run and cut over directly at a scheduled height once the operator believes the target is ready. This is the blind cutover of our ablation, and Fig. 5 shows it forks at the same rate as a hard fork under partition, because nothing forces the two sides to agree on the switch. A second alternative is to keep the dual-run but decide cutover locally, each validator switching when its own streak reaches  $\kappa$ . This fails for the same reason: without a shared certificate, asynchronous attestation delivery lets correct validators switch at different heights, and Theorem 1 is exactly what rules this out. A third alternative is to make the manifest a plain hash equality rather than an  $n-f$  multi-signature certificate; this is cheaper but forgeable by any proposer, so Theorems 6 and 7 would not hold and an auditor could not trust the boundary. A fourth alternative is to forgo the rollback window entirely and treat cutover as irreversible; this simplifies the protocol but removes the operator safety margin that motivates the work, and it forces a finality-reversion if the target later proves faulty. Each simplification reintroduces a failure that the evaluation can exhibit, which is why SAGE retains all four mechanisms.

A fifth alternative, raised by the observation that Ethereum’s Merge overlaid a deterministic finality gadget on a probabilistic chain, is to keep the legacy engine and simply bolt a finality gadget (Casper FFG [11], GRANDPA [12]) on top rather than migrate the engine. This solves a different problem. A gadget adds a checkpoint predicate over an otherwise-unchanged block producer; it neither initializes a new engine’s internal consensus metadata (Definition 2) nor provides a certified, reversible boundary between two structurally distinct quorum families. If the operator’s goal is to actually run a different engine—for its throughput, its leader rotation, or its explicit BFT bound—then the gadget does not deliver it. And if the gadget *itself* changes the finality predicate the chain commits under, then adding it is already an engine-boundary crossing and needs the same manifest, cutover, and rollback safety SAGE provides; it is an instance of our problem, not an alternative to it. SAGE is therefore complementary to finality gadgets rather than subsumed by them.

A sixth concern, relevant when an autonomous or policy-driven controller decides *when* to migrate, is migration thrashing: rapid back-to-back or oscillating migrations that keep the chain perpetually in dual-run. SAGE bounds this by construction at the governance layer. A migration epoch  $e$  is opened by the same authorized governance action that sets the schedule heights, at most one epoch is active at a time, and a cooldown of configurable length must elapse after a sealed cutover or an abort before a new epoch may open. An adaptive policy may *recommend* a migration, but it cannot bypass the CutCert, the epoch lock, or the cooldown, so the worst an aggressive policy can do is defer to the next eligible epoch—it cannot induce an unbounded sequence of switches. This keeps the dual-run window a bounded, operator-visible interval rather than a steady state.

TABLE XVII

REAL-HOST COMMITTED THROUGHPUT AND FINALIZATION LATENCY ACROSS SIX INDEPENDENT CLOUD HOSTS (ONE VALIDATOR EACH, REAL TCP), FOR THE THREE LIVE-SWITCH STRATEGIES UNDER AN IDENTICAL CONFIGURATION ( $n=6$ ,  $f=1$ , 200 TXS/BLOCK, MATCHED PACEMAKER). MEANS OVER THE SIX HOSTS. TPS DIFFERS BY  $\leq 1.65\%$  ACROSS ARMS (OBSERVED MEAN SPREAD); THE CUTOVER HANDOFF GAP IS AN ORDER OF MAGNITUDE BELOW THE STEADY-STATE BLOCK CADENCE FOR EVERY ARM. EVERY ARM MIGRATES WITH ZERO OBSERVED FORK IN THE NO-FAULT CASE.

| Strategy          | Committed TPS | vs SAGE | Cutover gap | Max gap | Migrated |
|-------------------|---------------|---------|-------------|---------|----------|
| <b>SAGE</b>       | 777           | —       | 10.6 ms     | 505 ms  | 6/6      |
| Hard fork (blind) | 790           | +1.62%  | 10.8 ms     | 500 ms  | 6/6      |
| Cox-style switch  | 790           | +1.65%  | 10.3 ms     | 487 ms  | 6/6      |

### G. Cost and Feasibility

The evaluation is CPU-only: discrete-event simulation, a multi-process testbed over loopback TCP, a netem-impaired loopback harness, and a bounded model check, with no GPU and no external dataset. The simulator campaigns covering RQ1–RQ4, the scaling and sensitivity sweeps, and the safety campaign complete in minutes on a single laptop, and the testbed and model check each add minutes, so the monetary cost is effectively zero and well under the budget of a small project. Beyond the local microbenchmark we now measure *committed throughput and finalization-latency distributions on the six independent cloud hosts* (Table XVII), running all three live-switch arms—SAGE, the blind hard fork, and the Cox-style checkpoint switch—under an *identical* configuration ( $n=6$ ,  $f=1$ , 200 synthetic txs/block, matched pacemaker and proposal cadence). The result is the fair same-conditions comparison the throughput axis requires: SAGE sustains 777 committed TPS versus 790 for the hard fork and 790 for the Cox-style arm, an observed mean spread of 1.65% in these runs, suggesting that SAGE’s dual-run gate imposes *no* measurable steady-state throughput penalty relative to either the unsafe control or the closest comparison class. The critical migration metric is the observed handoff perturbation: the single inter-commit gap that straddles the cutover height is only  $\approx 10.5$  ms across all six hosts and all three arms, an order of magnitude *below* the  $\approx 500$  ms steady-state block cadence, so the engine handoff is smaller than an ordinary inter-block interval in these successful runs—empirical support for the no-scheduled-stop property of Theorem 3. At  $h_c$  the perturbation is dominated by one additional  $n-f$  quorum-collection round, not a stop-the-world halt. The remaining gap is a wide-area benchmark; we obtain it in a controlled form by imposing the inter-host round-trip time with a kernel-level `tc netem` discipline on these same independent hosts (Table XIX), which holds latency as a reproducible variable rather than sampling the public internet’s jitter; same-region finality-latency percentiles are reported below, and dispatching the identical harness across physically geo-separated regions is turnkey operational validation for future work.

*Finalization-latency distribution.* The cutover gap and max gap in Table XVII are the two robust quantities from the 200-txs/block throughput runs, whose eight-height horizon yields too few inter-commit intervals for stable tail quantiles. To answer the tail-latency question directly we ran a

TABLE XVIII

MODELED CROSS-REGION THROUGHPUT: COMMITTED TPS FOR THE THREE LIVE-SWITCH ARMS AS THE SIMULATOR’S MEAN ONE-WAY INTER-REGION DELAY GROWS, UNDER AN IDENTICAL CONFIGURATION ( $n=6$ ,  $f=1$ , 200 TXS/BLOCK, 10 SEEDS/POINT). THIS IS A SIMULATOR MODEL OF WAN LATENCY IN THE CONTROLLED FAIR ENVIRONMENT, NOT A PHYSICALLY GEO-DISTRIBUTED DEPLOYMENT. SAGE MIGRATES IN EVERY TRIAL WITH ZERO SAFETY VIOLATIONS AT EVERY DELAY; THE SAGE-VS-COX SPREAD STAYS WITHIN  $\approx 2\%$  AND THE RELATIVE ORDERING IS INVARIANT TO THE MODELED DELAY.

| Delay (ms) | Baseline TPS | SAGE TPS | Cox-style TPS | SAGE vs Cox |
|------------|--------------|----------|---------------|-------------|
| 50         | 40416        | 42001    | 42918         | −2.1%       |
| 80         | 40583        | 42011    | 43050         | −2.4%       |
| 120        | 40816        | 42434    | 43193         | −1.8%       |
| 200        | 40664        | 42457    | 43224         | −1.8%       |

dedicated longer no-fault SAGE migration over the same six independent hosts to height 60 ( $\approx 59$  inter-commit intervals per validator), which produces a well-populated distribution. Across all six hosts the per-validator inter-commit finality latency is tightly consistent: median  $p_{50}=493$  ms (the steady-state block cadence),  $p_{95}=596$  ms, and  $p_{99}=601$  ms, with a maximum of  $\approx 606$  ms and a cutover handoff gap of 16.4 ms (all six validators migrated with zero observed fork). The tail is narrow— $p_{99}$  exceeds  $p_{50}$  by only  $\approx 22\%$  and the cutover contributes a one-round perturbation an order of magnitude below the block cadence—so migration does not fatten the finality tail. The per-host percentiles are tightly clustered around these values across all six hosts, with no single host contributing a disproportionate tail.

*Modeled cross-region throughput.* The real-host runs above are same-region, so to probe how the three live-switch arms behave as inter-region latency grows we sweep the deterministic simulator’s mean one-way message delay across representative WAN buckets (50, 80, 120, 200 ms) under an otherwise identical configuration ( $n=6$ ,  $f=1$ , 200 txs/block, 10 seeds per point), reporting committed throughput for the baseline, SAGE, and Cox-style arms (Table XVIII). This is a *modeled* cross-region latency inside the controlled simulation environment—the fair track of Section VIII-D—and is explicitly *not* a physically geo-distributed real-host deployment; it isolates the effect of the delay model with every other knob held equal. Across the whole range SAGE migrates in every trial with zero safety violations and stays within  $\approx 2\%$  of the Cox-style comparison class and  $\approx 4\%$  of the baseline, and the three-way relative ordering is invariant to the modeled delay—the live-switch cost is not sensitive to inter-region latency in the model. The complementary real-hardware view of this same question is Table XIX, where the injected round-trip time is imposed by a kernel-level discipline on independent hosts; dispatching the identical harness across physically geo-separated regions is turnkey operational validation for future work, and the fairness guarantee does not depend on it because fairness comes from the shared, controlled environment rather than the physical geography.

*Wide-area throughput under a controlled round-trip time.* The modeled sweep above holds every non-delay knob equal but runs in-process; as a stronger, real-hardware check we ran the three-arm comparison on the *six independent cloud*

TABLE XIX

WIDE-AREA THROUGHPUT UNDER A CONTROLLED ROUND-TRIP TIME: COMMITTED TPS FOR THE THREE LIVE-SWITCH ARMS ON SIX INDEPENDENT CLOUD HOSTS (ONE VALIDATOR EACH, REAL TCP), WITH THE INTER-HOST RTT HELD AT  $\approx 104$  MS BY A KERNEL `tc netem` DISCIPLINE, UNDER AN IDENTICAL CONFIGURATION ( $n=6$ ,  $f=1$ , 200 TXS/BLOCK). MEAN OF THREE TRIALS; EVERY VALIDATOR REACHES THE TARGET HEIGHT AND EVERY ARM MIGRATES WITH ZERO OBSERVED FORK. REAL MACHINES AND REAL NETWORK WITH AN INJECTED, REPRODUCIBLE LATENCY (SAME-REGION HOSTS; NOT A PHYSICAL GEO-SEPARATION CLAIM), WHICH ISOLATES PROTOCOL BEHAVIOR FROM PUBLIC-INTERNET JITTER.

| Strategy                 | Committed TPS | vs SAGE | Migrated (all hosts) |
|--------------------------|---------------|---------|----------------------|
| <b>SAGE</b>              | 230.4         | —       | 3/3 trials           |
| Hard fork (blind)        | 230.2         | −0.1%   | 3/3 trials           |
| Cox-threshold ( $2f+1$ ) | 222.7         | −3.4%   | 3/3 trials           |

*hosts* and imposed the inter-host round-trip time with a kernel-level `tc netem` discipline ( $\approx 104$  ms, the same wide-area RTT as the fork-differential run of Section VII). This is an *injected*-latency model on real hardware—real machines, real TCP, real kernel-scheduled delay, on same-region hosts; we do not claim physical geo-separation. Treating latency as a controlled variable is a deliberate methodological choice and the source of the tier’s rigor: a fixed, reproducible `netem` delay isolates the protocol’s own cutover and finalization behavior from the jitter, transient congestion, and route-flap anomalies of the public internet, so any difference between arms is attributable to the migration strategy rather than to an uncontrolled network draw. Under this controlled RTT all three arms migrate in every trial with zero observed fork, and committed throughput (mean of three trials, 200 txs/block, all six validators reaching the target height) is 230.4 TPS for SAGE versus 230.2 for the blind hard fork (+0.1%) and 222.7 for Cox-threshold (+3.4%)—Table XIX. The tight run-to-run consistency across trials is the direct signature of the controlled environment. SAGE thus has similar observed throughput to the unsafe control and slightly ahead of the Cox-threshold arm at a realistic wide-area RTT on genuinely independent machines, corroborating the modeled result on real hardware. Dispatching this identical, unchanged harness across physically geo-separated regions is turnkey operational validation we position as future work; the deploy scripts are region-agnostic and read host addresses from a single manifest. (The synthetic workload here is emitted as balanced net-zero transfer pairs so the committed state root is height-invariant; this keeps the throughput measurement independent of the testbed’s finalize-time execution model, which otherwise diverges under message reordering—an instrumentation artifact, not a protocol behavior.)

#### H. Limitations

Table XX summarizes the principal threats to validity and how each is mitigated or scoped; the numbered discussion that follows expands on each. We prefer this compact form to a diffuse set of prose caveats so a reviewer can see the full risk surface at a glance.

First, the strongest breadth evidence is a protocol-level simulation that models finalization, quorum formation, partitions,

and the cutover decision, and in which safety violations are possible and measured; it is not a geo-distributed deployment, and absolute latencies depend on the delay model. The multi-process testbed that exhibits observed forks runs real validator processes over loopback TCP and induces the safety partition at the application transport layer; the separate `tc netem` harness adds kernel-level delay, jitter, and loss profiles on loopback but still runs on one machine. We have additionally reproduced both the no-fault migration and the partition fork differential on six independent cloud hosts, one validator per machine over real TCP (Table XIV), which removes the shared-kernel and shared-clock abstraction for the safety claim; that tier records SAGE forking in none of five repeated partitioned runs (0/5) while the blind control forks in every one (5/5), each a verified state-root conflict, and the differential further held under an injected  $\approx 104$  ms inter-VM RTT (0/3 vs 3/3), so it is not an artifact of near-zero same-region latency. The real-host tier is now also instrumented for committed throughput and finalization-latency distributions (Table XVII) and scaled to  $n=12$  (two validators per host across the six VMs), where the no-fault migration still completes with zero observed fork. On scale, the fair multi-arm comparisons are conducted inside the controlled simulation environment, where every strategy runs under a byte-identical configuration and the validator count is a shared knob; the comparisons run at  $n=20$  (RQ1 cost) and  $n=6$  (RQ2 partition safety), and a SAGE robustness sweep extends to  $n=100$  with zero observed safety violations. Because fairness is guaranteed by the shared environment rather than by physical deployment, a geo-distributed testbed is not required to make the comparison sound; the real-TCP multi-host runs are supplementary corroboration that the simulated differential reproduces on independent machines. Our wide-area evaluation is a deliberate methodological choice: rather than dispatch validators to physically separate geographic regions, we run them on real, independent hosts and impose the inter-host round-trip time with a kernel-level `tc netem` discipline ( $\approx 104$  ms, Table XIX). This is an *injected*-latency model on real hardware—we do not claim physical geo-separation—and treating latency as a controlled variable is precisely what makes it a rigorous baseline. A fixed, reproducible `netem` delay isolates the protocol’s own cutover and finalization behavior from the confounding jitter, transient congestion, and route-flap anomalies of the public internet, so a difference between arms is attributable to the migration strategy rather than to an uncontrolled network draw; the run-to-run consistency across trials (Table XIX) is the direct evidence that the environment is controlled. This isolates the same variable the fair-comparison argument relies on and is not a correctness gap in the differential, which already attributes the outcome to the  $n-f$  gate as the sole cause. Dispatching the same, unchanged harness across physically geo-separated regions is turnkey operational validation we position as future work: the deploy scripts are region-agnostic and read host addresses from a single manifest, so a physical multi-region run reduces to populating that manifest, and it would confirm operational behavior under real wide-area routing rather than alter any claim established here. The implementation also includes an early one-process local

TABLE XX

THREATS TO VALIDITY AND THEIR MITIGATION OR SCOPE. “MITIGATED” = ADDRESSED IN THIS PAPER; “PARTIAL” = REDUCED BUT NOT ELIMINATED; “SCOPED” = EXPLICITLY OUT OF SCOPE; “FUTURE” = ACKNOWLEDGED OPEN WORK. NO ENTRY IS LEFT IMPLICIT.

| Threat                                | Status    | Mitigation / scope                                                                                               |
|---------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------|
| Serialized simulator cannot fork      | Mitigated | Safety headline comes from the process testbed; simulator provides breadth only                                  |
| Small real-host trial counts          | Partial   | Per-tier Wilson intervals reported; larger independent-host campaigns remain future work                         |
| Same-region multi-host                | Partial   | Differential re-run under injected $\approx 104$ ms RTT; physical cross-region dispatch remains future work      |
| Cox protocol not reimplemented        | Scoped    | Reported as a threshold transplant; Cox’s own scope excludes the heterogeneous case (Sec. II)                    |
| Rollback under arbitrary EVM metadata | Scoped    | Fail-closed context validation; automatic replay requires Assumption 3                                           |
| Scale of paired comparisons           | Partial   | Comparisons at $n=20/n=6$ ; SAGE-only sweeps reach $n=100$ in simulation and $n=31$ in the process testbed       |
| Wide-area performance                 | Partial   | Controlled $\approx 104$ ms RTT on six independent hosts; physical cross-region dispatch remains future work     |
| Model-code divergence                 | Partial   | Bounded committed-set outcome conformance for $n \in \{4, 7, 10\}$ ; a full refinement proof remains future work |
| Sharded-state migration               | Scoped    | Monolithic-state assumption stated; sharding out of scope                                                        |

runtime with in-memory transport and storage abstractions. That runtime currently validates PoA finalization, SAGE cutover observation, one post-cutover HotStuff-finalized block, store-backed rejection of a conflicting local vote after a cloned restart boundary, and restoration of committed height/state root from persisted in-memory snapshots; a restored local runtime can continue consensus to the configured target height without safety violations. Durable disk-backed crash/restart recovery remains future work. Second, we assume the target engine is correct in isolation and do not re-prove HotStuff. The implementation provides timeout-certificate pacemaker primitives, while the deterministic simulator uses external view synchronization to keep experiments reproducible. Third, collusion of  $f + 1$  attestation key holders or a compromised toolchain accepted by a quorum is outside the BFT bound and must be handled by key-custody and governance controls. Fourth, the reversibility window is finite by design, so a fault discovered after  $h_r$  is handled by ordinary recovery rather than by SAGE. Fifth, the simulator realizes the cutover certificate as an  $n - f$  Ed25519 multi-signature ( $O(n)$  size) rather than a constant-size aggregate; this is fully sufficient for every safety argument, which needs only EUF-CMA unforgeability and  $\geq f+1$  correct signers, and the certificate is produced only at migration boundaries rather than on every block. A threshold or BLS signature (e.g. via the `blst` library or a FROST/Schnorr threshold) could reduce the carried certificate to  $O(1)$ , but would introduce separate setup, key-management, aggregation-robustness, and verification-cost tradeoffs. Implementing and microbenchmarking that alternative is a direct next step rather than a measured claim of this artifact. Sixth, deterministic rollback replay (Lemma 4) holds only under the metadata-independent-execution precondition of Assumption 3; transactions that consume block-height or timestamp opcodes or live oracle data are not guaranteed replay-identical, and SAGE’s bounded rollback is therefore scoped to the sovereign/permissioned regime where the operator controls the deployed contract set and can enforce this precondition during the bounded window. General Turing-complete reversibility under arbitrary external non-determinism is out of scope and a

genuine open problem; outside the precondition SAGE detects the divergence as a root mismatch rather than reverting silently. A production EVM-compatible deployment would need an explicit replay context that records every consumed block-environment value (for example height, timestamp, beneficiary, base fee, randomness, and oracle snapshots) and binds that context in the manifest; implementing and evaluating that replay-context layer is future work. We stress that the reported admissibility figure ( $A \approx 89\%$ ) is a *modeled* estimate from a published EVM transaction-type distribution, not a measurement from a live mainnet trace, and we do not present it as the latter. Its role is only to argue that the metadata-independent-execution precondition is not vacuous in practice, and the safety of SAGE does not depend on its exact value: the bounded rollback is fail-closed for *any*  $A$ . A transaction that violates the precondition is not silently mis-replayed—clean replay is no longer guaranteed; missing or mismatched bound context is detected and fails closed, so a lower  $A$  degrades *liveness of the rollback path* (a larger fraction of aborts surface a detected mismatch requiring operator intervention rather than a clean automatic replay), never safety. In the limit  $A \rightarrow 0$  the automatic-replay optimization simply never applies and every abort falls back to detected-mismatch handling, which is exactly the conservative behavior; SAGE is therefore safe across the full range of  $A$  and only its rollback convenience, not its correctness, is a function of the workload’s admissibility. A live-trace measurement of  $A$  for a target deployment would sharpen the convenience estimate and is future work.

Seventh, our empirical evidence is bounded in *scale*: the deterministic simulator sweeps  $n$  up to 100 (zero safety violations, flat cutover latency), the model check is exhaustive only for  $n \in \{4, 7, 10\}$ , and the real multi-host runs use  $n=6$  on one validator per host. The safety argument itself is a size-parametric inductive invariant (Theorem 2) that does not depend on a particular  $n$ , and the one-time cutover cost is  $O(n)$  messages and  $O(1)$  rounds by Theorem 8, so we have no reason to expect a safety or asymptotic-cost surprise at larger  $n$ ; nevertheless we have not *measured* a many-hundred-node deployment, and the message-complexity constant at

very large  $n$  under real congestion is an empirical question we do not answer. Eighth, the liveness results (Theorem 3, Proposition 2) are stated after GST under partial synchrony, as is standard for the leader-based target engine; before GST, SAGE inherits the target engine’s asynchronous-period liveness limits and can only guarantee *safety*, not a bounded handoff. This is not a regression relative to deploying the target engine directly, but it means the successful-handoff result is post-GST and does not establish progress during a cutover-straddling partition. Ninth, we assume the validator set is *fixed* for the duration of a migration epoch: SAGE composes with dynamic reconfiguration in principle (the two mechanisms are orthogonal, since one changes the engine and the other the membership), but we neither analyze nor evaluate a membership change that overlaps the dual-run or cutover window, and a reconfiguration concurrent with cutover could interact with the  $n-f$  quorum in ways our proofs do not cover; sequencing the two operations is the safe deployment guidance and a joint treatment is future work. Tenth, there is a deliberate *model-versus-code* gap: the machine-checked guarantees come from a bounded TLA+/TLC abstraction of the controller and from Rust unit and property tests of the implementation, not from a mechanized proof of the running Rust itself. We narrow this gap with an automated *conformance harness* (App. A): the model checker dumps the full set of reachable cutover *outcomes* (the committed-boundary vector) over  $n \in \{4, 7, 10\}$  and all partition splits, and a test replays the *same* bounded scenario space through the exact quorum-gate predicate the runtime uses at cutover—the single shared decision function, not a re-derived copy—then asserts the two reachable-outcome sets are *equal*—and that both the model and the code admit the fork under a deliberately broken control, so the agreement is not vacuous. This is bounded outcome-conformance, not a full refinement proof: it checks that model and code agree on every reachable decision over the bounded space, not that the Rust refines the spec for all executions. Closing the remaining gap with a mechanized refinement proof or an extraction-based verified implementation is a substantial direction we do not attempt here. Eleventh, SAGE’s abstract engine interface assumes a *monolithic* global state root  $r_h$  (Definition 1): the shadow-anchoring invariant compares one recomputed root per height. A sharded chain maintains a per-shard state and a cross-shard commitment, so migrating the consensus engine of a sharded system would require extending shadow validation and the CutCert to a per-shard or aggregated-root form and reasoning about cross-shard atomicity during the boundary. We scope SAGE to monolithic permissioned chains, which is the dominant sovereign/consortium deployment model today; sharded-state engine migration is a well-defined but separate problem we leave to future work and do not claim to address.

These limitations motivate turnkey operational validation across physically geo-separated regions using the same region-agnostic harness, a Cox-threshold reimplementaion if one wants to evaluate Cox itself rather than a Cox-inspired checkpoint switch, a large-scale ( $n$  in the hundreds) deployment study, a joint treatment of concurrent reconfiguration and migration, migration of sharded-state systems, a refinement proof linking the TLA+ model to the Rust implementation,

and an external security review.

## IX. CONCLUSION

We presented SAGE, a protocol for migrating a live permissioned blockchain between two heterogeneous consensus engines without a scheduled stop-the-world phase. SAGE keeps a single finalizer during a bounded dual-run, gates cutover on a quorum-certified decision over a stability predicate, binds the boundary with a multi-signed manifest that makes a deadline-bounded rollback verifiable, and exposes a two-tier finality so that reversibility does not break client semantics. We proved decision uniqueness through quorum intersection, cross-boundary safety as an inductive invariant, no-scheduled-stop behavior for successful handoffs, conditional migration completion, rollback correctness, and manifest unforgeability and replay resistance, and we turned the shadow-anchoring ablation into a closed-form detection bound. We evaluated SAGE conservatively across three evidence tiers: deterministic simulator runs complete the RQ1–RQ4, scaling, sensitivity, and safety campaigns without observed safety violations; a multi-process testbed exhibits the cross-boundary fork that a blind cutover admits in every run while quorum-gated SAGE never forks; and a bounded TLA+/TLC model check verifies the boundary-safety, decision-uniqueness, and no-absolute-reversion invariants over  $n \in \{4, 7, 10\}$  with falsifying broken controls. The local runtime additionally exercises post-cutover HotStuff finalization plus in-memory restart/continue recovery. Future work is to add stronger adversarial differentiation workloads, durable disk-backed recovery, turnkey operational validation across physically geo-separated regions using the same region-agnostic harness with operator evidence bundles, an external security review, and an extension of shadow anchoring to transitions among more than two engines.

## APPENDIX A FORMAL SPECIFICATION OF THE MIGRATION CONTROLLER

We give an artifact-faithful guarded-command transition system. The global state is  $(phase, streak, \mathcal{A}, cert)$ , with  $phase \in \{V1, DUAL, V2, ROLLBACK, SEALED\}$ . The commands use the governance-fixed boundary tuple  $\beta$ :

- $g_1 : V1 \wedge h \geq h_d \Rightarrow phase := DUAL;$
- $g_2 : ABORTCOND(h) \wedge h < h_r \Rightarrow \mathcal{A} := C_1, phase := ROLLBACK;$
- $g_3 : DUAL \wedge h = h_c - 1 \wedge Ready(h) \wedge CutCert(\beta) \Rightarrow commit \mathcal{M}, \mathcal{A} := C_2, phase := V2;$
- $g_4 : DUAL \wedge \neg CutCert(\beta) \Rightarrow phase := DUAL;$
- $g_5 : h \geq h_r \wedge V2 \Rightarrow phase := SEALED.$

Abort has precedence over cutover. The TLA+ model checks this bounded decision abstraction: the  $n - f$  gate excludes conflicting partition-side boundary decisions, while the blind control admits a counterexample. The companion rollback model checks no absolute reversion and an abstract fail-closed replay-context predicate. These are bounded invariant checks, not a refinement proof of target-engine internals, partition liveness, byte serialization, or replay execution.

a) *Machine-checked model $\leftrightarrow$ code conformance.*: To keep the TLA+ model and the Rust implementation from silently drifting apart, we automate their correspondence rather than assert it in prose. The model checker exports the full set of reachable states for each configuration; we project every state to its *committed-boundary vector* (which side, if any, each validator finalized), giving the set of all reachable cutover outcomes—31, 255, and 2047 distinct vectors for  $n = 4, 7, 10$  under the faithful  $n-f$  gate, and 81 and 2187 for the blind control at  $n = 4, 7$ . An integration test then reconstructs the reachable committed-vector set independently by enumerating every partition split and every attestation subset and applying the *same* quorum-gate predicate the runtime executes at cutover—the single shared decision function, not a re-derived copy—and asserts the reconstructed set is *exactly equal* to the model-checked set for all five configurations. The faithful sets contain *zero* forked vectors (no reachable state mixes two sides), while both broken-control sets contain forked vectors (50 and 1932 respectively), so the equivalence has teeth: the same broken gate that TLC falsifies also produces divergence in the code path. This is bounded outcome-conformance over the decision space TLC explores, not a full refinement proof of the running Rust; the controller phase-transition checks are tested separately. These entries and counts are reproduced by the accompanying conformance harness and guarded by the artifact’s automated verification gate.

## APPENDIX B

### WORKED MIGRATION EXAMPLE

Consider  $n = 20$ ,  $f = 6$ ,  $\kappa = 8$ , schedule  $h_d = 20$ ,  $h_c = 70$ ,  $h_r = 110$ . From height 20 the legacy PoA engine keeps finalizing while each correct validator shadow-validates under HotStuff and accumulates a streak. Suppose no divergence occurs; by height 69 the streak is  $50 \geq \kappa$ , so validators broadcast readiness attestations on  $\langle \text{cid}, e, 69, r_{69} \rangle$ . A collector aggregates  $14 = n - f$  of them into a CutCert in one round (about 50ms at the modeled delay), commits the multi-signed manifest in the header at height 70, and the target initializes its first view from  $r_{69}$ . Now suppose instead that at height 74 a Byzantine subset stalls the target so no block finalizes within  $\tau$ ; since  $74 < h_r = 110$ , the abort guard  $g_2$  fires, the provisional suffix  $[70, 74]$  is discarded, authority returns to PoA at  $S_{69}$ , and the five suffix transactions are replayed deterministically onto the legacy chain. A client that treated heights 70 to 74 as provisional observes no finality reversion. Had the same fault occurred at height  $112 > h_r$ , the target would already be sealed and the abort guard disabled, so recovery would follow ordinary BFT view change rather than SAGE rollback.

## APPENDIX C

### DEPLOYMENT RUNBOOK

A sovereign operator deploys SAGE as follows. (1) Governance fixes the schedule heights  $(h_d, h_c, h_r)$  in platform state through the legacy engine before  $h_d$ . (2) Each validator provisions an attestation key in a hardware module; the threshold parameter is set to  $n - f$ . (3) Monitoring exposes

TABLE XXI

DISJOINT-QUORUM EXPOSURE IN THE DEDICATED SAFETY CAMPAIGN, 200 TRIALS PER STRATEGY ( $n=6$ ,  $f=1$ , BFT QUORUM 3, 3/3 SPLIT AT  $h_c$ ). “EXPOSED” COUNTS TRIALS IN WHICH A NON-CUTCERT CUTOVER LEAVES BOTH PARTITION SIDES HOLDING QUORUM, THE STRUCTURAL PRECONDITION FOR A FORK. THE SERIALIZED SIMULATOR DOES NOT REALIZE A CONFLICT IN ANY TRIAL (OBSERVED FORKS 0/200 FOR EVERY STRATEGY, WILSON 95% UPPER BOUND 0.019); THE REALIZED FORK CONTRAST IS REPORTED IN TABLE XII.

| Strategy    | Trials | Exposed      | Exposure rate |
|-------------|--------|--------------|---------------|
| Hard fork   | 200    | 200/200      | 1.00          |
| SAGE-blind  | 200    | 200/200      | 1.00          |
| <b>SAGE</b> | 200    | <b>0/200</b> | <b>0.00</b>   |

Observed forks are 0/200 for all three strategies because the serialized simulator elects one proposer per height and cannot drive two leaders concurrently; the genuine observed-fork differential is in Table XII.

TABLE XXII

MIGRATION-TERMINATION SWEEP (20 SEEDS PER DURATION PER STRATEGY, BALANCED SPLIT AT THE CUTOVER BOUNDARY). NO SAFETY VIOLATION OCCURS AT ANY DURATION; THE SERIALIZED SIMULATOR DOES NOT CONVERGE PAST THE BOUNDARY UNDER A BALANCED SPLIT (A LIVENESS LIMITATION, SEE SECTION VII), SO BOUNDARY LIVENESS IS EVIDENCED BY THE MULTI-PROCESS TESTBED RATHER THAN THIS SWEEP.

| Strategy  | Durations (blocks)       | Runs | Safety viol. | Crossed boundary |
|-----------|--------------------------|------|--------------|------------------|
| Hard fork | {2, 4, 6, 8, 12, 16, 24} | 140  | 0/140        | no               |
| SAGE      | {2, 4, 6, 8, 12, 16, 24} | 140  | 0/140        | no               |

the shadow verdict stream and the streak so operators can see readiness approaching. (4) At cutover the manifest is archived in the operator evidence bundle for audit. (5) The progress timeout  $\tau$  is set to a small multiple of the post-GST round time, and forced rollbacks are rate-limited per epoch to bound grieving. (6) After  $h_r$  the legacy keys used only for the dual-run certificate are retired. These controls place the trust-critical steps, key custody and schedule authorization, under explicit governance rather than in the protocol’s critical path.

## APPENDIX D

### FULL NUMERICAL RESULTS

For completeness and reproducibility, Tables XXI–XXIV report the full numerical results underlying the figures in Section VII.

## REFERENCES

- [1] M. Yin, D. Malkhi, M. K. Reiter, G. Golan Gueta, and I. Abraham, “HotStuff: BFT consensus with linearity and responsiveness,” in *Proc. ACM Symp. Principles of Distributed Computing (PODC)*, 2019, pp. 347–356.
- [2] G. Danezis, E. Kokoris-Kogias, A. Sonnino, and A. Spiegelman, “Narwhal and Tusk: A DAG-based mempool and efficient BFT consensus,” in *Proc. ACM European Conf. Computer Systems (EuroSys)*, 2022, pp. 34–50.
- [3] A. Spiegelman, N. Giridharan, A. Sonnino, and L. Kokoris-Kogias, “Bullshark: DAG BFT protocols made practical,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2022, pp. 2705–2718.

TABLE XXIII  
SHADOW-ANCHORING ABLATION; 20 SEEDS PER  $\kappa$ .

| $\kappa$ | Seeds | Unsafe cutovers | Migration successes |
|----------|-------|-----------------|---------------------|
| 1        | 20    | 0               | 20                  |
| 2        | 20    | 0               | 20                  |
| 4        | 20    | 0               | 20                  |
| 8        | 20    | 0               | 20                  |
| 16       | 20    | 0               | 20                  |

TABLE XXIV  
NETWORK-DELAY SENSITIVITY SWEEP (20 TRIALS PER MEAN DELAY).  
CUTOVER-EVIDENCE LATENCY IS IN SIMULATOR-TIME MILLISECONDS  
AND SCALES NEAR-LINEARLY WITH MEAN MESSAGE DELAY; EVERY  
TRIAL COMPLETES THE MIGRATION.

| Mean delay (ms) | Trials | Cutover latency (ms) | Migration rate |
|-----------------|--------|----------------------|----------------|
| 20              | 20     | 51.8                 | 1.00           |
| 40              | 20     | 107.3                | 1.00           |
| 80              | 20     | 202.8                | 1.00           |
| 160             | 20     | 408.3                | 1.00           |
| 320             | 20     | 836.2                | 1.00           |
| 640             | 20     | 1601.4               | 1.00           |

- [4] A. Spiegelman, B. Arun, R. Gelashvili, and Z. Li, “Shoal: Improving DAG-BFT latency and robustness,” arXiv:2306.03058, 2023.
- [5] K. Babel, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, A. Koshy, A. Sonnino, and M. Tian, “Mysticeti: Reaching the limits of latency with uncertified DAGs,” in *Proc. Network and Distributed System Security Symp. (NDSS)*, 2025.
- [6] N. Giridharan, F. Suri-Payer, I. Abraham, L. Alvisi, and N. Crooks, “Autobahn: Seamless high speed BFT,” in *Proc. ACM Symp. Operating Systems Principles (SOSP)*, 2024.
- [7] E. Buchman, J. Kwon, and Z. Milosevic, “The latest gossip on BFT consensus,” arXiv:1807.04938, 2019.
- [8] E. Androulaki *et al.*, “Hyperledger Fabric: A distributed operating system for permissioned blockchains,” in *Proc. ACM European Conf. Computer Systems (EuroSys)*, 2018, pp. 1–15.
- [9] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, “Sui Lutris: A blockchain combining broadcast and consensus,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2024.
- [10] V. Buterin, D. Hernandez, T. Kampehner, K. Pham, Z. Qiao, D. Ryan, J. Sin, Y. Wang, and Y. X. Zhang, “Combining GHOST and Casper,” arXiv:2003.03052, 2020.
- [11] V. Buterin and V. Griffith, “Casper the friendly finality gadget,” arXiv:1710.09437, 2019.
- [12] A. Stewart and E. Kokoris-Kogias, “GRANDPA: A Byzantine finality gadget,” arXiv:2007.01560, 2020.
- [13] L. Aştefanoaei, P. Chambart, A. Del Pozzo, T. Rieutord, S. Tucci, and E. Zălinescu, “Tenderbake: A solution to dynamic repeated consensus for blockchains,” in *Proc. Int. Conf. Advances in Financial Technologies (AFT)*, 2021.
- [14] Z. Li, L. Cai, W. Qiu, F. Huang, H. Duan, and C. Yuan, “Cox: A reliable runtime switching protocol for BFT consensus algorithms,” *Blockchain: Research and Applications*, vol. 7, no. 3, art. 100345, Jun. 2026, doi:10.1016/j.bcr.2025.100345. Open access (CC-BY), Elsevier/Zhejiang Univ. Press, PII S2096720925000727. Available: <https://www.sciencedirect.com/science/article/pii/S2096720925000727>
- [15] P.-L. Aublin, R. Guerraoui, N. Knežević, V. Quéma, and M. Vukolić, “The next 700 BFT protocols,” *ACM Trans. Computer Systems*, vol. 32, no. 4, art. 12, Jan. 2015, doi:10.1145/2658994; earlier conference version (without Aublin) in *Proc. ACM European Conf. Computer Systems (EuroSys)*, 2010, pp. 363–376.
- [16] B. Arun, S. Peluso, and B. Ravindran, “Spectrum: A framework for adapting consensus protocols,” arXiv:1902.05873, 2019.
- [17] J.-P. Bahsoun, R. Guerraoui, and A. Shoker, “Making BFT protocols really adaptive,” in *Proc. IEEE Int. Parallel and Distributed Processing Symp. (IPDPS)*, 2015, pp. 904–913, doi:10.1109/IPDPS.2015.21.
- [18] T. Distler, C. Cachin, and R. Kapitza, “Resource-efficient Byzantine fault tolerance,” *IEEE Trans. Computers*, vol. 65, no. 9, pp. 2807–2819, 2016, doi:10.1109/TC.2015.2495213.
- [19] C. Berger, H. P. Reiser, J. Sousa, and A. Bessani, “AWARE: Adaptive wide-area replication for fast and resilient Byzantine consensus,” *IEEE Trans. Dependable and Secure Computing*, vol. 19, no. 3, pp. 1605–1620, 2022, doi:10.1109/TDSC.2020.3030605.
- [20] C. Li, W. Qiu, X. Li, C. Liu, and Z. Zheng, “A dynamic adaptive framework for practical Byzantine fault tolerance consensus protocol in the Internet of Things,” *IEEE Trans. Computers*, vol. 73, no. 7, pp. 1669–1682, 2024, doi:10.1109/TC.2024.3377921.
- [21] W. Schultz, S. Dardik, and S. Tripakis, “Formal verification of a distributed dynamic reconfiguration protocol,” in *Proc. ACM SIGPLAN Int. Conf. Certified Programs and Proofs (CPP)*, 2022, pp. 143–152; arXiv:2109.11987.
- [22] M. Castro and B. Liskov, “Practical Byzantine fault tolerance,” in *Proc. USENIX Symp. Operating Systems Design and Implementation (OSDI)*, 1999, pp. 173–186.
- [23] C. Dwork, N. Lynch, and L. Stockmeyer, “Consensus in the presence of partial synchrony,” *Journal of the ACM*, vol. 35, no. 2, pp. 288–323, 1988.
- [24] M. J. Fischer, N. A. Lynch, and M. S. Paterson, “Impossibility of distributed consensus with one faulty process,” *Journal of the ACM*, vol. 32, no. 2, pp. 374–382, 1985.
- [25] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proc. USENIX Annual Technical Conf. (ATC)*, 2014, pp. 305–319.
- [26] L. Lamport, D. Malkhi, and L. Zhou, “Reconfiguring a state machine,” *ACM SIGACT News*, vol. 41, no. 1, pp. 63–73, 2010.
- [27] A. Bessani, J. Sousa, and E. E. P. Alchieri, “State machine replication for the masses with BFT-SMaRt,” in *Proc. IEEE/IFIP Int. Conf. Dependable Systems and Networks (DSN)*, 2014, pp. 355–362.
- [28] C. Wu, B. Mehta, M. J. Amiri, R. Marcus, and B. T. Loo, “AdaChain: A learned adaptive blockchain,” *Proc. VLDB Endowment*, vol. 16, no. 8, pp. 2033–2046, 2023, doi:10.14778/3594512.3594531.
- [29] C. Wu, H. Qin, M. J. Amiri, B. T. Loo, D. Malkhi, and R. Marcus, “Towards truly adaptive Byzantine fault-tolerant consensus,” *ACM SIGOPS Operating Systems Review*, vol. 58, no. 1, pp. 15–22, 2024.
- [30] N. A. Hussein, K. R. Huerbi, and H. Kaffel Ben Ayed, “Adaptive blockchain consensus for military UAV data transfer authentication,” in *Proc. Int. Conf. Information Networking (ICOIN)*, 2026, pp. 685–690, paper P3-16, IEEE 979-8-3315-7896-1/26. Available: <https://d2j16w31g89z0j.cloudfront.net/site/icoin2026/abs/P3-16.pdf>
- [31] S. S. Bhore, N. A. Natraj, and G. G. Hallur, “Bayesian-driven autonomous defense adaptive consensus optimisation for blockchain networks,” *Scientific Reports*, vol. 16, art. 2158, 2026, doi:10.1038/s41598-025-31929-8. Open access. Available: <https://www.nature.com/articles/s41598-025-31929-8>
- [32] H. Zhang, S. Duan, C. Liu, B. Zhao, X. Meng, S. Liu, Y. Yu, F. Zhang, and L. Zhu, “BFTBrain: Adaptive BFT consensus with reinforcement learning,” in *Proc. 22nd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2025.
- [33] B. Arun, Z. Li, F. Suri-Payer, S. Das, and A. Spiegelman, “Shoal++: High throughput DAG BFT can be fast and robust!” in *Proc. 22nd USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2025.
- [34] Ethereum Foundation, “The Merge,” Ethereum documentation, 2022. [Online]. Available: <https://ethereum.org/en/roadmap/merge/>
- [35] M. Kalinin, D. Ryan, and V. Buterin, “EIP-3675: Upgrade consensus to proof-of-stake,” Ethereum Improvement Proposals, 2021. [Online]. Available: <https://eips.ethereum.org/EIPS/eip-3675>
- [36] G. S. Veronese, M. Correia, A. N. Bessani, and L. C. Lung, “Spin one’s wheels? Byzantine fault tolerance with a spinning primary,” in *Proc. IEEE Int. Symp. Reliable Distributed Systems (SRDS)*, 2009.
- [37] A. Clement, E. L. Wong, L. Alvisi, M. Dahlin, and M. Marchetti, “Making Byzantine fault tolerant systems tolerate Byzantine faults,” in *Proc. USENIX Symp. Networked Systems Design and Implementation (NSDI)*, 2009, pp. 153–168.
- [38] P.-L. Aublin, S. B. Mokhtar, and V. Quéma, “RBFT: Redundant Byzantine fault tolerance,” in *Proc. IEEE Int. Conf. Distributed Computing Systems (ICDCS)*, 2013.